BHAGWAN PARSHURAM INSTITUTE OF TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING - DATA SCIENCE

AAROGYASATHI: AI-DRIVEN PUBLIC HEALTH CHATBOT FOR DISEASE AWARENESS

A MINOR PROJECT DISSERTATION

submitted in partial fulfillment of the requirement for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering - Data Science

Submitted by:

SALMAN MOHAMMAD KHAN (Enrollment No. 00520815723)
DHAIRYA SUMMI (Enrollment No. 00420815723)
SIDHARTH SINGH (Enrollment No. 01820815723)

Under the Supervision of:

Mr. Aditya Sam Koshy
Assistant Professor
Department of Computer Science and Engineering

Department of Computer Science & Engineering - Data Science
Bhagwan Parshuram Institute of Technology
Affiliated to Guru Gobind Singh Indraprastha University, Delhi
PSP-4, Sector-17, Rohini, Delhi - 110089

NOVEMBER 2026

BHAGWAN PARSHURAM INSTITUTE OF TECHNOLOGY
VISION AND MISSION OF THE INSTITUTE

VISION OF THE INSTITUTE

- To establish a leading Global Center of Excellence in multidisciplinary education, training and research in the area of Engineering, Technology and Management.
- To produce technologically competent, morally & emotionally strong and ethically sound professionals who excel in their chosen field, practice commitment to their profession and dedicate themselves to the service of mankind.

MISSION OF THE INSTITUTE

- To develop world class Laboratories and other Infrastructure conducive in acquiring latest knowledge and expertise.
- To bridge the knowledge and competency gaps of institute's fresh pass-outs vis-à-vis field requirements.
- To strengthen Industry-Institute Interaction and partnership for imbibing corporate culture amongst our faculty and students.
- To promote research culture among faculty and students enhancing their academic and professional confidence needed to face global challenges.
- To honor commitment towards social and moral values.

PROGRAM OUTCOMES (POs), COURSE OUTCOMES (COs), AND SUSTAINABLE DEVELOPMENT GOALS (SDGs)

PROGRAM OUTCOMES (POs)

PO 1. Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, and engineering fundamentals and an engineering specialization to solve complex engineering problems.

PO 2. Problem Analysis: Identify, formulate, and analyze complex engineering problems, reaching substantiated conclusions considering holistic aspects.

PO 3. Design/development of Solutions: Design creative solutions for complex engineering problems and design systems/components that meet identified needs with consideration for public health and safety, cultural, societal, and environmental factors.

PO 4. Conduct Investigations of Complex Problems: Conduct investigations using research-based knowledge and methods, including design of experiments, data analysis, and synthesis to provide valid conclusions.

PO 5. Engineering Tool Usage: Create, select, and apply appropriate techniques, modern engineering and IT tools, prediction, and modeling to solve complex problems, recognizing limitations.

PO 6. The Engineer and the World: Analyze and evaluate societal and environmental aspects while solving complex problems, considering impacts on economy, health, safety, legal frameworks, and environment.

PO 7. Ethics: Apply ethical principles and commit to professional ethics, human values, diversity, and inclusion; adhere to relevant laws.

PO 8. Individual and Collaborative Team Work: Function effectively as an individual and as a member or leader in diverse and multidisciplinary teams.

PO 9. Communication: Communicate effectively and inclusively with the engineering community and society, writing effective reports, documentation, and presentations.

PO 10. Project Management and Finance: Apply engineering management principles and economic decision-making to manage multidisciplinary projects.

PO 11. Life-long Learning: Recognize the need for, and have the ability for independent and lifelong learning in the broadest context of technological change.

COURSE OUTCOMES (COs) - MINOR PROJECT (ES-451)

CO 1: Apply engineering knowledge and design principles to identify, formulate, and solve a complex problem through a structured project.

CO 2: Conduct systematic investigation and analysis using modern tools and techniques for designing and implementing the Project.

CO 3: Demonstrate professional documentation, effective oral communication, and presentation skills through the dissertation.

CO 4: Work ethically as an individual or in a team to manage the Project effectively, considering societal, environmental, and sustainability aspects.

BHAGWAN PARSHURAM INSTITUTE OF TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING - DATA SCIENCE
DEPARTMENT VISION, MISSION, PEOs AND PSOs

VISION OF THE DEPARTMENT

To be a leading centre of excellence in the field of Computer Science Engineering in the realms of Data Science education, research, and innovation, committed to empowering students to lead data-driven decision-making for the betterment of society.

MISSION OF THE DEPARTMENT

- To deliver comprehensive education in Data Science by integrating core principles of Computer Science with modern analytical and computational tools to build competent and industry-ready professionals.
- To cultivate self-driven learners capable of recognizing and responding to modern societal and technological needs through research, higher studies, or entrepreneurship.
- To empower students with critical thinking and problem-solving skills to lead and innovate in Data Science and related fields.
- To foster meaningful industry engagement for the enrichment of all stakeholders.
- To instil ethical data practices through the development of professional integrity, social responsibility, and a lifelong commitment to learning in students.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO 1. Professional Excellence and Adaptability: Graduates will establish themselves as successful professionals, researchers, or entrepreneurs in Data Science by applying analytical, computational, and domain-specific knowledge to solve real-world problems while adapting to emerging technologies.

PEO 2. Ethics, Social Responsibility, and Leadership: Graduates will uphold ethical standards and social responsibility in data handling and decision-making while demonstrating leadership and teamwork in multidisciplinary environments.

PEO 3. Collaborative Engagement and Social Interaction: Graduates will communicate effectively and collaborate with diverse teams, nurturing productive community engagement.

PROGRAM SPECIFIC OUTCOMES (PSOs) (w.e.f 2021)

PSO 1: Apply core concepts of Computer Science and Data Science to develop effective and scalable solutions by leveraging data collection, processing, visualization, statistical modeling, and machine learning for real-world problems.

PSO 2: Design and develop sustainable data-driven intelligent systems using engineering principles and modern tools, while embracing lifelong learning to adapt to emerging technologies.

PSO 3: Graduates will demonstrate ethical and legal awareness in data usage, along with strong communication skills, enabling them to succeed as professionals, entrepreneurs, or in higher studies.

CO–PO AND CO–PSO MAPPING WITH JUSTIFICATIONS

Table 1: CO–PO Correlation Matrix for Minor Project

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	2	2	2	1	1	1	1	1	1	1
CO2	2	3	3	2	3	1	1	0	1	1	2
CO3	1	2	1	1	2	0	0	1	2	3	1
CO4	0	0	0	0	0	3	3	3	2	1	1

Project-Specific Justification for CO–PO Mapping:

• **CO1 (Apply knowledge and design principles):** Mapped strongly to PO1 (3) through the mathematical formulation of character n-gram TF-IDF representations, logistic regression intent extraction, and cosine similarity ranking with Language-agnostic BERT Sentence Embeddings (LaBSE). Substantial mapping to PO2-PO4 (2) through problem decomposition of rural infodemics and empirical evaluation across 49 clinical records, 13 vaccine milestones, and 64 Indian myth entries.

• **CO2 (Systematic investigation and implementation):** High correlation with PO2 (3), PO3 (3), and PO5 (3) through the design and deployment of the hybrid conversational pipeline, integration of scikit-learn 1.9.0, PyTorch 2.6.0, Transformers 5.15.1, Ollama local inference (Qwen3:1.7b), FastAPI ASGI architecture, and Meta WhatsApp Cloud API v25.0 webhook routing over Cloudflare secure tunnels on an Ubuntu ARM64 Oracle Cloud VM.

• **CO3 (Documentation and communication):** Strongly mapped to PO10 (3) and PO9 (2) through structured engineering documentation, architecture diagrams, sequence models, data flow models, and formal dissertation synthesis adhering strictly to college engineering formatting and APA reference standards.

• **CO4 (Ethics, teamwork, and sustainability):** Directly correlates with PO6 (3), PO7 (3), and PO8 (3) through adherence to WHO ethical guidelines for health AI, rigorous implementation of non-diagnostic clinical safety disclaimers, emergency triage escalation (108/112 hotlines), limited retention of personal data, collaborative peer pair programming, and git version-controlled task division.

CO–PSO MAPPING TABLE AND JUSTIFICATIONS

Table 2: CO–PSO Correlation Matrix for Minor Project

Course Outcome	PSO 1	PSO 2	PSO 3
CO1: Identify, formulate, and design data-driven solutions	3	2	2
CO2: Systematic investigation and modern tool implementation	2	3	2
CO3: Professional technical documentation and presentation	1	2	3
CO4: Ethical data handling, teamwork, and sustainability	1	2	3

Project-Specific Justification for CO–PSO Mapping:

• **CO1 to PSO1 (Level 3):** CO1 directly applies core Computer Science and Data Science principles to engineer multilingual NLP pipelines, combining lexical term-frequency modeling and dense semantic embeddings to resolve public health inquiries.

• **CO2 to PSO2 (Level 3):** CO2 involves end-to-end implementation and deployment of an intelligent conversational system using modern cloud platforms (Oracle Cloud Infrastructure ARM64, FastAPI, HuggingFace Transformers, Ollama, and Meta Graph API).

• **CO3 to PSO3 (Level 3):** CO3 emphasizes professional scientific writing, technical precision, and communication of complex AI architectural principles through structured engineering documentation.

• **CO4 to PSO3 (Level 3):** CO4 instills ethical data practices, clinical disclaimer integration, user privacy safeguards (ephemeral session state without persistent PII storage), and social responsibility in public health communication.

PROJECT TO SUSTAINABLE DEVELOPMENT GOAL (SDG) MAPPING

PROJECT TITLE: AarogyaSathi: AI-Driven Public Health Chatbot for Disease Awareness

Table 3: Alignment with United Nations Sustainable Development Goals

Mapped Sustainable Development Goals	Relevance Level
SDG 3: Good Health and Well-Being	Primary Direct Target (High)
SDG 10: Reduced Inequalities	Secondary Target (Substantial)
SDG 9: Industry, Innovation, and Infrastructure	Technological Target (Moderate)

Justification for SDG Mapping:

• **SDG 3: Good Health and Well-Being (Target 3.3, 3.d):** AarogyaSathi directly contributes to Target 3.3 (combating communicable diseases such as dengue, malaria, and tuberculosis) and Target 3.d (strengthening early warning, risk reduction, and management of national health risks). By disseminating verified WHO/MoHFW preventive guidance, validating viral misinformation, and routing potential emergencies directly to national emergency dispatch lines (108/112), the project promotes community well-being and health literacy.

• **SDG 10: Reduced Inequalities (Target 10.2, 10.3):** In rural and semi-urban India, socio-economic and linguistic barriers severely limit access to qualified healthcare professionals. AarogyaSathi democratizes health information access by providing cross-lingual conversational assistance across 12 Indian languages, breaking literacy barriers through ubiquitous mobile channels like WhatsApp and lightweight browser interfaces, ensuring that vulnerable populations receive the same verified health facts as urban communities.

• **SDG 9: Industry, Innovation, and Infrastructure (Target 9.b):** The project leverages open-source AI infrastructure (FastAPI, PyTorch, HuggingFace LaBSE, and Ollama) deployed on cloud computing infrastructure (Oracle Cloud ARM64), demonstrating how cost-effective, decentralized digital public infrastructure (DPI) can be developed and sustained in developing nations without commercial cloud dependency.

DECLARATION

This is to certify that the material embodied in this Minor Project - Dissertation titled 'AarogyaSathi: AI-Driven Public Health Chatbot for Disease Awareness' being submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering - Data Science is based on our original work. It is further certified that this Minor Project - Dissertation work has not been submitted in full or in part to this or any other university for the award of any other degree or diploma. Our indebtedness to other works has been duly acknowledged at the relevant places.

_____

(Salman Mohammad Khan)
Enrollment No. 00520815723

_____

(Dhairya Summi)
Enrollment No. 00420815723

_____

(Sidharth Singh)
Enrollment No. 01820815723

CERTIFICATE

This is to certify that the work embodied in this Minor Project - Dissertation titled 'AarogyaSathi: AI-Driven Public Health Chatbot for Disease Awareness' being submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering - Data Science, is original and has been carried out by Salman Mohammad Khan (Enrollment No. 00520815723), Dhairya Summi (Enrollment No. 00420815723), and Sidharth Singh (Enrollment No. 01820815723) under my supervision and guidance.

It is further certified that this Minor Project - Dissertation work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma to the best of my knowledge and belief.

(Mr. Aditya Sam Koshy) Faculty Supervisor Assistant Professor Department of Computer Science & Engineering Bhagwan Parshuram Institute of Technology	(Dr. Monika Arora) Incharge - CSE-DS Department of CSE-DS Bhagwan Parshuram Institute of Technology

ACKNOWLEDGEMENT

An endeavor is not complete and successful till the people who made it possible are given due credit for their guidance and support. We take this opportunity to express our profound gratitude to all those who have contributed to the successful completion of this Minor Project Dissertation.

At the very onset, we express our heartfelt gratitude to our faculty supervisor, Mr. Aditya Sam Koshy, Assistant Professor, Department of Computer Science and Engineering, for providing scholarly guidance, technical direction, and continuous encouragement throughout the formulation, implementation, and evaluation of AarogyaSathi. His profound insights into software architecture, machine learning methodologies, and rigorous verification standards kept our work grounded in engineering reality and steered us in the right direction.

We extend our sincere thanks to Prof. Payal Pahwa, Principal, Bhagwan Parshuram Institute of Technology, and Dr. Monika Arora, Incharge, Department of Computer Science & Engineering - Data Science, for providing institutional infrastructure, state-of-the-art facilities, and continuous academic encouragement to undertake challenging multidisciplinary research.

Last but not least, we convey our deepest gratitude to our parents, family members, and peers for their unconditional moral support, patience, and understanding throughout the course of our project work.

Salman Mohammad Khan (00520815723)
Dhairya Summi (00420815723)
Sidharth Singh (01820815723)
B.Tech (CSE-DS), 7th Semester

ABSTRACT

AarogyaSathi (आरोग्यसाथी) is an AI-driven, multilingual public health conversational assistant engineered to provide verified preventive healthcare guidance, viral health-claim verification, national immunization schedule tracking, and localized syndromic surveillance. Developed in response to Smart India Hackathon problem statement SIH25049 under the Government of Odisha's healthcare technology theme, the system addresses the severe infodemic of health misinformation and the acute scarcity of qualified clinical guidance in rural and semi-urban India. AarogyaSathi implements a multi-tier hybrid architecture combining deterministic rule-based triage, rule-based fast-path intent extraction, Language-agnostic BERT Sentence Embeddings (LaBSE) for cross-lingual dense semantic retrieval, and retrieval-augmented generation (RAG) powered by a locally hosted Qwen3 (1.7B parameter) language model alongside Ollama runtime inference pathways. The entire pipeline is deployed on an Oracle Cloud Infrastructure virtual machine (2 OCPUs, 12 GiB RAM, Ubuntu 24.04 ARM64) serving a FastAPI application, reached through a Cloudflare HTTPS tunnel, and accessible via a lightweight responsive web portal and a Meta WhatsApp Cloud API test-number channel (outbound API v25.0).

System reliability was evaluated through 37 automated unit and regression tests run on the deployment VM on 26 October 2026, covering routing, session caching, ML classifiers and surveillance anomaly detection—all passing; five WhatsApp webhook security tests added later also passed. Component benchmark evaluations demonstrated 30/30 (100%) exact intent classification, 15/15 (100%) top-1 LaBSE retrieval accuracy, and 34/34 (100%) concordance on categorical health-claim verification (mean latency 387 ms). However, rigorous evaluation uncovered critical engineering constraints: 14 of 30 intent test inputs duplicated training examples; lexical red-flag matching caused a negated phrase ('no chest pain, only a mild headache') to falsely trigger emergency ambulance escalation; a month-to-year conversion defect mapped 2-month infants to the 9–12 month vaccination schedule; and regional Odia vernacular generation dropped into a safe fallback. A Meta WhatsApp test-number demonstration answered three test messages from an approved recipient with no sending errors recorded in the server logs. AarogyaSathi demonstrates the feasibility of a low-cost public health information service on free-tier cloud infrastructure while establishing transparent boundaries for clinical safety, negation handling, and verified data linkage.

Keywords: *Public Health Informatics, Multilingual Conversational Agents, Retrieval-Augmented Generation (RAG), LaBSE Dense Embeddings, Health Misinformation Verification, Meta WhatsApp Cloud API, Syndromic Surveillance.*

TABLE OF CONTENTS

(Development Based Project)

Vision and Mission of the Institute ..... **ii**
Program Outcomes (POs), Course Outcomes (COs), and SDGs ..... **iii**
Department Vision, Mission, PEOs and PSOs ..... **iv**
CO–PO and CO–PSO Mapping with Justifications ..... **v**
Project to SDG Mapping with Justifications ..... **vii**
Declaration ..... **viii**
Certificate ..... **ix**
Acknowledgement ..... **x**
Abstract ..... **xi**
List of Figures ..... **xiv**
List of Tables ..... **xv**
CHAPTER 1: INTRODUCTION **1**
1.1 Healthcare Information Landscape and the Infodemic 1
1.2 Target Audience and Socio-Economic Context in Rural India 1
1.3 Project Motivation and Clinical Scope Boundaries 2
1.4 Smart India Hackathon Brief (SIH25049) and Objectives 2
1.5 Overview of the AarogyaSathi Solution Architecture 3
1.6 Background and Literature Review 4
1.7 Organization of the Report 5
CHAPTER 2: PROBLEM STATEMENT **6**
2.1 Formal Problem Definition 6
2.2 Core Project Objectives 6
2.3 Key Stakeholders and User Personas 7
2.4 System Boundaries, Operating Assumptions, and Clinical Non-Claims 8
2.5 Acceptance Criteria and Success Metrics 9
CHAPTER 3: ANALYSIS **10**
3.1 Software Requirement Specifications (SRS Overview) 10
3.2 Functional Requirements (FR-1 through FR-8) 10
3.3 Non-Functional Requirements (NFR-1 through NFR-6) 11
3.4 Feasibility Study (Technical, Operational, Economic) 11
3.5 Tools, Technologies, and Hardware Platform 12
3.6 Use Case Modeling and Actor Interactions 13
3.7 Data Flow Modeling (Level 0 and Level 1 DFD) 14
CHAPTER 4: DESIGN AND ARCHITECTURE **16**
4.1 System Architecture and Work Breakdown Structure (WBS) 16
4.2 Detailed Module Responsibilities 17
4.3 Data Storage Architecture: SQLite vs. Curated JSON vs. Sessions 18
4.4 Knowledge Base Schema and Domain Taxonomy 19

4.5 Cross-Lingual Semantic Retrieval and Dense Vector Search....20
4.6 Fallback Intent Classification Taxonomy....20
4.7 Multi-Tier Triage, Fast-Path Heuristics, and Routing Flow....21
4.8 Health Misinformation Verification Engine....22
4.9 Syndromic Telemetry and Outbreak Surveillance Modeling....23
4.10 Meta WhatsApp Cloud API Channel Sequence Flow....24
CHAPTER 5: IMPLEMENTATION....25
5.1 System Implementation Overview and Service Topology....25
5.2 Core Router and Fast-Path Heuristic Implementation....25
5.3 Knowledge Base Retrieval and Dense Vector Search Implementation....26
5.4 Misinformation Verification Engine Implementation....26
5.5 Meta WhatsApp Webhook Security and Handler Implementation....27
5.6 Web Conversational Interface Implementation....27
5.7 Conversational Walkthrough and Real-World Query Execution....27
CHAPTER 6: TESTING AND QUALITY ASSURANCE....31
6.1 Comprehensive Verification and Testing Strategy....31
6.2 Unit and Regression Test Suite Execution....31
6.3 Component Benchmark Evaluation (Intent, Retrieval, Fact-Check)....32
6.4 Synthetic End-to-End Query Spot Checks and Clinical Defect Analysis....33
6.5 Code Inspection Findings: Vaccination Age Conversion Defect....34
6.6 Meta WhatsApp Cloud API Channel Live Verification....35
6.7 SIH25049 Challenge Compliance Matrix....35
CHAPTER 7: SUMMARY AND CONCLUSION....37
7.1 Summary of Delivered Work and Technical Achievements....37
7.2 Significance for Public Health Information Dissemination....38
7.3 Concluding Remarks....38
CHAPTER 8: LIMITATIONS AND FUTURE WORK....39
8.1 Identified Architectural and Clinical Safety Limitations....39
8.2 Geographic and Epidemiological Surveillance Modeling Constraints....39
8.3 Linguistic and Regional Knowledge Base Gaps....40
8.4 Deployment, Privacy, and Channel Constraints....40
8.5 Prioritized Strategic Roadmap for Future Enhancements....40
BIBLIOGRAPHY....41
APPENDIX A: PRODUCTION HARDWARE & ENVIRONMENT SPECIFICATIONS 43
APPENDIX B: INTENT CLASSIFICATION TAXONOMY & API DIRECTORY.....43

LIST OF FIGURES

Figure 1.1: High-Level Solution Topology and Channel Architecture of AarogyaSathi.....3
Figure 3.1: Use Case Model for Citizen, Health Worker, and Administrative Actors.....13
Figure 3.2: Context-Level Data Flow Diagram (DFD Level 0) for Public Health Inquiries. .14
Figure 3.3: Detailed Functional Data Flow Diagram (DFD Level 1) of Core Engine.....15
Figure 4.1: Work Breakdown Structure (WBS) of AarogyaSathi Engineering Modules.....16
Figure 4.2: Data Storage Architecture: SQLite Telemetry vs. Curated JSON vs. RAM Sessions.....18
Figure 4.3: Activity and Decision Flowchart of Multi-Tier Triage and Fast-Path Routing....22
Figure 4.4: Meta WhatsApp Cloud API Integration Sequence Flow and Webhook Ingress. .24
Figure 5.1: Web Chat Landing View and Disease-Information Answer (Dengue).....28
Figure 5.2: Fact-Check Verdict (MYTH) and Hindi Answer for a Malaria Query.....29
Figure 5.3: Administrative Syndromic Surveillance Dashboard Populated with Demonstration Data.....30
Figure 5.4: Meta Developer Console Webhook Fields Showing the Subscribed Messages Field.....30

LIST OF TABLES

Table 1.1: Comparative Analysis of Conventional Chatbots vs. AarogyaSathi Hybrid Pipeline 5
Table 3.1: Functional Requirements Specification (FR-1 through FR-8).....10
Table 3.2: Non-Functional Requirements Specification (NFR-1 through NFR-6).....10
Table 3.3: Minimum Hardware and Software Execution Environment Specifications.....12
Table 4.1: Curated Knowledge Base Inventory Across Public Health Domains.....19
Table 4.2: Intent Classification Taxonomy and Training Corpus Distribution.....21
Table 6.1: Automated Test Suite Coverage (37 Tests Run on 26 October 2026; 5 Channel Security Tests Added on 31 October 2026).....32
Table 6.2: Component Benchmark Evaluation Results on Deployed Host.....32
Table 6.3: Selected Live Synthetic Query Spot Checks and Clinical Response Observations 34
Table 6.4: Meta WhatsApp Cloud API Channel Live Verification Summary.....35
Table 6.5: SIH25049 Challenge Compliance Matrix: Brief Requirements vs. Prototype Status 36
Table 8.1: Prioritized Strategic Roadmap and Clinical Validation Matrix for Future Work...40
Table A.1: Oracle Cloud VM Production Hardware and Environment Topology.....43
Table B.1: AarogyaSathi Production REST API Endpoint Directory.....43

CHAPTER 1
INTRODUCTION

1.1 Healthcare Information Landscape and the Infodemic

In contemporary public health administration, the rapid proliferation of digital communication networks has fundamentally altered how citizens seek, interpret, and act upon health-related information. While internet connectivity has expanded dramatically across developing nations, the sheer volume of unfiltered digital content has precipitated what the World Health Organization (WHO) formally classifies as an infodemic—an overabundance of information, both accurate and inaccurate, that accompanies an acute health emergency or pervades routine healthcare management (World Health Organization et al., 2020). In India, where mobile internet penetration exceeds 800 million broadband subscribers, digital messaging platforms such as WhatsApp have become the primary conduit for informal communication. However, this ubiquitous reach is accompanied by profound risks: unverified home remedies for vector-borne diseases (e.g., claims that raw garlic or papaya leaf extracts eliminate dengue viruses), vaccine hesitancy stoked by baseless assertions connecting immunizations to neurological disorders, and delayed hospital presentations caused by misdiagnosed acute symptoms, translating directly into preventable morbidity and severe economic stress.

1.2 Target Audience and Socio-Economic Context in Rural India

The primary beneficiaries of this project are citizens residing in rural, peri-urban, and tribal districts of India, with specific initial contextualization toward the state of Odisha under the Smart India Hackathon 2025 mandate. In these geographic regions, healthcare infrastructure is characterized by a severe density deficit of qualified allopathic clinicians, where the doctor-to-population ratio frequently falls significantly below the WHO-recommended threshold of 1:1,000. Rural primary health centres (PHCs) and community health centres (CHCs) operate under severe diagnostic constraints, placing enormous responsibility on frontline community health workers—Accredited Social Health Activists (ASHAs) and Auxiliary Nurse Midwives (ANMs). Compounding this deficit are formidable linguistic and digital literacy barriers across 22 scheduled languages. Monolingual English health portals

fail completely when accessed by rural citizens communicating exclusively in regional idioms such as Odia or Hindi, creating an urgent societal imperative for an accessible, voice- and text-compatible vernacular conversational interface.

1.3 Project Motivation and Clinical Scope Boundaries

AarogyaSathi (आरोग्यसाथी, translating to 'Healthcare Companion') was conceived as an intelligent, automated digital public health assistant engineered to bridge the gap between authoritative clinical knowledge and grassroots citizen inquiries. The core motivation is to empower every citizen with immediate, low-cost access to medically grounded disease awareness, immunization tracking, and health rumor verification, without requiring personal travel or financial sacrifice. Crucially, the system is designed with rigorous and unambiguous clinical scope boundaries: AarogyaSathi is strictly an informational and educational health awareness tool; it is explicitly not a diagnostic system, a medical triage substitute, or an automated prescribing platform (Semigran et al., 2015). Routine queries receive educational guidance accompanied by mandatory clinical disclaimers, while potential emergencies immediately trigger hardcoded red-flag escalations directing users to national emergency hotlines (108/112).

1.4 Smart India Hackathon Brief (SIH25049) and Objectives

The project was formulated under the auspices of Smart India Hackathon 2025, specifically addressing problem statement SIH25049 released by the Government of Odisha under the Healthcare Technology theme (DRIEMS University, 2025). The official challenge brief mandated developing an AI-driven, multilingual public health chatbot capable of disseminating disease prevention information, symptoms, national immunization schedules, outbreak warnings, and health-forward verification via web, WhatsApp, and SMS channels. In response, five technical objectives were established:

- **Objective 1 (Curated Knowledge Grounding):** Construct an authoritative structured knowledge base encompassing major endemic diseases, childhood immunization milestones, and rural health inquiries grounded strictly in WHO, MoHFW, and ICMR guidelines.

- **Objective 2 (Cross-Lingual Semantic Retrieval):** Implement a cross-lingual retrieval pipeline using Language-agnostic BERT Sentence Embeddings (LaBSE) to support queries in regional Indian languages without multi-stage translation errors.
- **Objective 3 (Automated Misinformation Verification):** Engineer an automated fact-checking engine matching forwarded claims against curated myth repositories to assign categorical truth verdicts with transparent clinical rationales.
- **Objective 4 (Ubiquitous Multi-Channel Ingress):** Deploy the solution across a lightweight web portal and an automated Meta WhatsApp Cloud API channel, ensuring friction-free access on basic smartphones.
- **Objective 5 (Syndromic Anomaly Surveillance):** Incorporate localized geographic telemetry based on citizen symptom submissions and postal pincodes for rolling 7-day syndromic outbreak detection.

1.5 Overview of the AarogyaSathi Solution Architecture

To achieve these objectives within strict operational constraints, AarogyaSathi implements a multi-tier hybrid architecture (Figure 1.1). Incoming messages from web or WhatsApp ingress points first traverse deterministic safety and fast-path heuristic filters. Queries exhibiting acute red-flag terminology bypass all machine learning inference and immediately return urgent medical dispatch directives. Routine health inquiries undergo rule-based fast-path pattern extraction; if a known disease profile or immunization milestone is identified, pre-compiled verified templates are dispatched instantly, avoiding generative-model latency for routine requests.

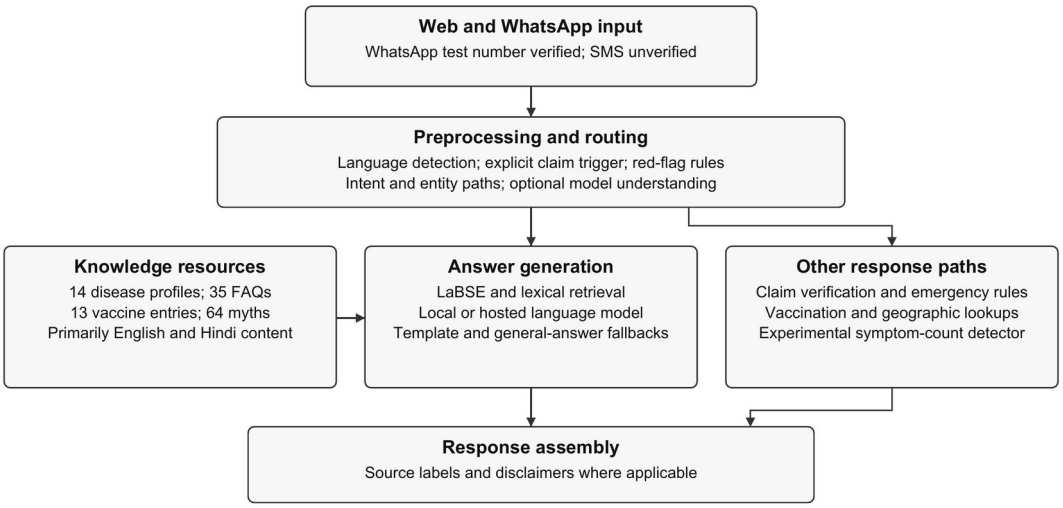


Figure 1.1: High-Level Solution Topology and Channel Architecture of AarogyaSathi (Adapted from Khan et al., 2026).

When queries exhibit complex syntax, the system engages its dense vector semantic retrieval pipeline. The query is embedded via a 768-dimensional LaBSE transformer, scored against indexed records using a composite cosine-lexical similarity function, and injected as grounded context into a locally hosted Qwen3 (1.7B parameter) large language model running via Ollama. By constraining generation to retrieved clinical records, the architecture minimizes hallucination while preserving fluent vernacular expression. Simultaneously, symptom submissions are aggregated into SQLite for rolling seven-day syndromic cluster detection.

1.6 Background and Literature Review

The conceptual foundation of AarogyaSathi builds upon Retrieval-Augmented Generation (RAG; Lewis et al., 2020) and cross-lingual representation via Language-agnostic BERT Sentence Embeddings (LaBSE; Feng et al., 2022). RAG pairs parametric neural generators with non-parametric dense vector document indices, dramatically reducing factual hallucination while allowing domain knowledge to be updated without model retraining. LaBSE maps semantically equivalent sentences across 109 languages into a shared embedding space, allowing colloquial Hindi queries to exhibit high cosine similarity with English clinical disease records without translation pipeline latency.

However, medical conversational literature urges extreme caution. Systematic reviews by Laranjo et al. (2018) and Milne-Ives et al. (2020) revealed that the majority of medical chatbots were evaluated on small developer-curated dialogue subsets without independent clinical validation, blinded review, or safety testing under adversarial negation. Semigran et al. (2015) audited symptom checkers and discovered that triage advice was appropriate in only 57% of cases. These empirical findings directly informed our testing philosophy: rather than claiming unsubstantiated clinical perfection, AarogyaSathi's evaluation explicitly highlights failure modes, training overlap, and negation vulnerabilities. Table 1.1 provides a comparative feature summary:

Table 1.1: Comparative Analysis of Conventional Chatbots vs. AarogyaSathi Hybrid Pipeline

Feature Dimension	Conventional Commercial Bots	Generic Generative LLMs	AarogyaSathi Hybrid Pipeline
Knowledge Provenance	Static rule trees or closed API	Parametric web training memory	Curated WHO/MoHFW Grounded Store
Cross-Lingual Support	Machine translated English	Multilingual token completion	Native LaBSE Dense Vector Space
Misinformation Checking	Not supported / out of scope	Variable parametric hallucination	Dedicated 64-Myth Verification Engine
Emergency Triage	Generic disclaimer	Varies with prompt engineering	Hardcoded Deterministic Red-Flag Bypass
Operating Cost	High SaaS per-user licensing	Expensive token API metering	Open-source stack on free-tier hosting
Channel Accessibility	Proprietary standalone app	Web browser interface only	Dual: Lightweight Web + Meta WhatsApp

1.7 Organization of the Report

The remainder of this dissertation is organized into seven subsequent chapters conforming strictly to the BPIT B.Tech Minor Project format. Chapter 2 formalizes the Problem Statement, defining core objectives, stakeholder personas, and system boundaries. Chapter 3 presents the Analysis phase, detailing the Software Requirements Specification (SRS), functional and non-functional requirements, feasibility studies, use cases, and Data Flow Diagrams. Chapter 4 outlines the Design and Architecture, covering the Work Breakdown Structure (WBS), module responsibilities, database schemas, LaBSE retrieval mathematics, and WhatsApp sequence models. Chapter 5 elaborates upon the Implementation, analyzing key algorithms, service topologies, and providing a visual screenshot walkthrough of web and WhatsApp channels. Chapter 6 details Testing and Quality Assurance, reviewing the automated test suite, component benchmarks, training overlap analysis, clinical spot-check defects, and Meta WhatsApp live delivery. Chapter 7 provides the Summary and Conclusion. Finally, Chapter 8 transparently articulates Project Limitations and a Prioritized Strategic Roadmap for Future Work, followed by an APA Bibliography and Technical Appendices.

CHAPTER 2
PROBLEM STATEMENT

2.1 Formal Problem Definition

The fundamental problem addressed by this project can be formally stated as follows: Rural and semi-urban populations in India suffer from an acute lack of timely, linguistically accessible, and clinically verified healthcare information, resulting in widespread vulnerability to viral health misinformation, delayed medical care presentation, low childhood vaccination compliance, and uncontrolled seasonal epidemic surges.

This overarching problem decomposes into three distinct structural challenges:

- **1. The Vernacular Accessibility Barrier:** While official public health guidelines issued by the Ministry of Health and Family Welfare (MoHFW) and the World Health Organization (WHO) are comprehensive, they are overwhelmingly published in dense, academic English or formal bureaucratic Hindi. Over 70% of rural Indian citizens communicate in regional languages and vernacular dialects, rendering official digital health advisories functionally inaccessible.
- **2. Uninhibited Proliferation of Digital Health Misinformation:** Social messaging platforms facilitate the frictionless, viral transmission of pseudoscientific health claims. Forwarded messages advocating unproven herbal cures for vector-borne diseases or stoking vaccine hesitancy circulate without any automated fact-checking mechanism, actively undermining national immunization programs and public health interventions.
- **3. Critical Clinical Latency and Triage Failures:** Citizens experiencing early-stage symptoms often struggle to determine whether their condition warrants routine home care, an outpatient primary health clinic visit, or immediate emergency ambulance escalation. Generic internet search engines return overwhelming lists of potential diagnoses ranging from benign fatigue to terminal malignancies, causing unnecessary panic or dangerous complacency.

2.2 Core Project Objectives

To systematically address the formal problem statement, AarogyaSathi establishes five precise, measurable technical objectives:

- **Objective 1: Multi-Tier Hybrid Conversational Pipeline:** Architect and deploy a high-performance conversational engine capable of resolving citizen inquiries via deterministic safety rules, rule-based fast-path heuristics, and dense vector semantic retrieval, with model-generated answers returned within a few seconds on the cloud virtual machine.
- **Objective 2: Authoritative Multilingual Knowledge Grounding:** Curate and maintain an authoritative, structured digital knowledge base covering 14 major endemic diseases, 13 national childhood vaccination milestones, and 35 frequent health inquiries in English and Hindi, with semantic cross-lingual detection across 12 Indian languages.
- **Objective 3: Heuristic Misinformation Fact-Checking:** Develop a dedicated claim-verification engine capable of ingesting forwarded text claims, performing cross-lingual similarity matching against a curated catalog of 64 Indian health myths, and returning transparent, categorical verdicts (VERIFIED, MYTH, PARTLY TRUE, UNVERIFIABLE) with clinical rationales.
- **Objective 4: Ubiquitous Dual-Channel Mobile Ingress:** Implement dual interfaces: a responsive, zero-install mobile web application and a Meta WhatsApp Cloud API integration (demonstrated with an approved test recipient), enabling citizens to interact with the chatbot on basic smartphones without incurring data or subscription costs.
- **Objective 5: Geospatial Syndromic Outbreak Telemetry:** Engineer an experimental geographic surveillance module that aggregates voluntary, anonymous citizen symptom submissions by postal pincode and evaluates rolling 7-day statistical anomaly baselines to detect potential syndromic disease clusters.

2.3 Key Stakeholders and User Personas

To ensure human-centered engineering, the system was designed around three illustrative personas (composite examples, not real individuals) representing key stakeholders in India's public healthcare delivery chain:

- **Persona 1: Ramesh (Rural Smallholder Farmer, Mayurbhanj, Odisha):** Ramesh is 42 years old, speaks Odia, and possesses basic functional literacy in Hindi. He accesses the internet exclusively through a low-cost Android smartphone with prepaid 4G data. His family lives 18 kilometers from the nearest Primary Health Centre. When his son develops acute fever, chills, and muscle soreness, Ramesh needs immediate, trusted advice in simple Hindi or Odia to

determine whether it could be malaria and what immediate steps must be taken, without being misled by WhatsApp remedies.

- **Persona 2: Sunita Devi (ASHA Community Health Worker, Rural Cuttack):** Sunita is an Accredited Social Health Activist responsible for monitoring maternal and child health across two village hamlets (population ~1,800). She requires an authoritative, instant reference tool on her smartphone to verify exact national immunization milestones for infants at 6, 10, and 14 weeks, and to debunk prevalent village rumors regarding measles-rubella vaccine safety during community immunization drives.
- **Persona 3: Dr. Patnaik (District Surveillance Officer, IDSP Odisha):** Dr. Patnaik oversees communicable disease monitoring across the district. He requires automated digital tools capable of identifying unusual spikes in reported syndromic symptoms (such as acute febrile illness or loose stools) mapped by postal pincode, providing early indicator telemetry days before formal lab-confirmed cases are logged in the Integrated Disease Surveillance Programme (IDSP) portal.

2.4 System Boundaries, Operating Assumptions, and Clinical Non-Claims

A foundational requirement of this dissertation is establishing transparent boundaries regarding what the system can and cannot perform. AarogyaSathi operates strictly under the following system boundaries and assumptions:

- **1. Non-Diagnostic Posture:** AarogyaSathi does not provide differential diagnoses, does not issue clinical prescriptions, and does not calculate pharmaceutical dosages. All generated responses include mandatory medical disclaimers emphasizing that information is for awareness only and cannot substitute for an examination by a registered medical practitioner.
- **2. Ephemeral Session Telemetry:** To limit data retention, the persistent SQLite database stores only pincode, symptom, intent, language and verdict fields, and no message text, names or phone numbers. Short conversational history is held in volatile memory with a 30-minute Time-To-Live (TTL) and is discarded when the session expires.
- **3. Outbreak Advisory Illustrative Nature:** While the system features an operational syndromic cluster detection algorithm, its bundled outbreak records are illustrative seasonal advisories designed for demonstration purposes, and do

not represent an authenticated, synchronized live feed from state or central government epidemiologists.

- **4. Restricted Demonstration Environment:** The Meta WhatsApp Cloud API channel operates under an approved developer test-number deployment (+1 555 626 0618) restricted to authorized recipient phone numbers, rather than an unconstrained commercial enterprise production number.

2.5 Acceptance Criteria and Success Metrics

The project's engineering completion was governed by six rigorous acceptance criteria:

- **AC-1 (Test Suite Integrity):** 100% pass rate across the comprehensive automated test suite (spanning core routing, RAG retrieval, ML classifiers, and channel adapters) run on the deployment host.
- **AC-2 (Fast-Path Latency):** Deterministic rule-based replies (greetings, emergency red flags) are returned without invoking the language model, and model-generated answers are returned within a few seconds on the cloud virtual machine.
- **AC-3 (Retrieval Precision):** Top-1 retrieval accuracy exceeding 90% across curated clinical test queries evaluated against the 49-record indexed health corpus.
- **AC-4 (Fact-Checking Coverage):** Accurate categorical truth assignment across a curated test suite of common Indian public health myths without unhandled runtime exceptions.
- **AC-5 (Channel Operational Verification):** Verified bidirectional message transmission and webhook signature validation across both the web application and the Meta WhatsApp Cloud API test number.
- **AC-6 (Clinical Safety Red-Flag Assurance):** Deterministic interception and urgent hotline escalation for critical emergency phrases, ensuring safety paths bypass all probabilistic language models.

CHAPTER 3
ANALYSIS

3.1 Software Requirement Specifications (SRS Overview)

The Software Requirement Specification (SRS) for AarogyaSathi defines the complete functional and non-functional capabilities of the system, establishing the technical baseline against which the software was engineered, deployed, and evaluated. The system is designed as an asynchronous, service-oriented web and messaging platform optimized for high concurrency, low latency, and free-tier hosting on cloud infrastructure.

3.2 Functional Requirements (FR)

The functional scope of AarogyaSathi is organized into eight primary requirement modules (FR-1 through FR-8), detailed in Table 3.2:

Table 3.1: Functional Requirements Specification (FR-1 through FR-8)

Req ID	Functional Module	Detailed Requirement Specification
FR-1	Multilingual Ingress & Language Detection	The system shall ingest free-text messages via Web and WhatsApp webhooks, detect user language across 12 Indian languages, and maintain session context in volatile memory.
FR-2	Deterministic Emergency Triage	The system shall immediately intercept predefined red-flag symptoms (chest pain, acute dyspnea, severe hemorrhaging) and return emergency hotline dispatch directives (108/112), bypassing ML pipelines.
FR-3	Fast-Path Rule Execution	The system shall match routine queries (greetings, standard disease inquiries, vaccine ages) using compiled regex patterns to return verified responses without invoking the generative model.
FR-4	Dense Vector Semantic Retrieval	The system shall embed unresolved health inquiries using LaBSE (768-dim vectors) and retrieve the top-k matching clinical profiles from the indexed corpus using cosine-lexical reranking.
FR-5	Retrieval-Augmented Generation	The system shall inject retrieved clinical context into an open-source language model (Qwen3:1.7b) running via Ollama to synthesize concise, grounded advice in the user's language.
FR-6	Health Misinformation Fact-Checking	The system shall process forwarded claims prefixed with 'fact check:' or matching known rumors, compare them against 64 curated Indian myths, and return categorized verdicts with source citations.

FR-7	Syndromic Anomaly Detection	The system shall log voluntary citizen symptom-pincode pairs into SQLite and trigger automated severity alerts when 24-hour counts exceed 3x the 7-day rolling historical baseline.
FR-8	Meta WhatsApp Cloud API Integration	The system shall receive webhook payloads from Meta Graph API v25.0, cryptographically verify HMAC-SHA256 signatures, deduplicate message IDs, and dispatch split replies via HTTPS.

3.3 Non-Functional Requirements (NFR)

In addition to functional capabilities, the architecture enforces six critical non-functional engineering standards (Table 3.2):

Table 3.2: Non-Functional Requirements Specification (NFR-1 through NFR-6)

Req ID	Quality Attribute	Target Specification & Evaluation Criterion
NFR-1	Latency & Performance	Rule-based replies are returned without invoking the language model; model-assisted answers are returned within a few seconds; webhook requests are acknowledged before background processing completes.
NFR-2	Privacy & Ethical Security	No message text, names or phone numbers stored in the database; ephemeral in-memory sessions with 30-min TTL; HMAC-SHA256 signature validation on webhooks.
NFR-3	Reliability & Availability	Continuous operation via systemd services on Oracle VM; automatic service recovery on unexpected process termination; graceful degradation if Ollama fails.
NFR-4	Hosting Resource Efficiency	Operational within 2 OCPUs and 12 GiB RAM on an Oracle Cloud Arm virtual machine without specialized GPU hardware acceleration.
NFR-5	Linguistic Adaptability	Cross-lingual semantic understanding across English, Hindi, and 10 regional Indian languages via LaBSE multilingual embeddings.
NFR-6	Clinical Grounding & Safety	Mandatory inclusion of clinical disclaimers on every non-emergency output; citation of verified public health authorities (WHO, MoHFW, NVBDCP).

3.4 Feasibility Study (Technical, Operational, Economic)

Before architectural commitment, a comprehensive three-dimensional feasibility study was conducted:

- **1. Technical Feasibility:** Deployment of transformer embeddings and quantized language models on ARM64 cloud architecture was evaluated. The LaBSE transformer and Qwen3:1.7b under Ollama's C++ runtime operate within 3.5 GiB active memory, well below the 12 GiB threshold of our Oracle Cloud VM.

Asynchronous FastAPI request handling ensures high concurrency without worker starvation.

- **2. Operational Feasibility:** In rural India, citizens are heavily accustomed to WhatsApp and lightweight mobile browsers. By deploying AarogyaSathi directly over WhatsApp and responsive HTML5 web interfaces, little installation or onboarding friction is introduced, which fits existing rural communication habits.
- **3. Economic Feasibility:** The solution relies exclusively on open-source software (Python 3.12, FastAPI, PyTorch, Ollama, SQLite) and free-tier infrastructure (an Oracle Cloud Arm virtual machine, a Cloudflare HTTPS tunnel, and the Meta WhatsApp Developer test number). No recurring software or API license fees were incurred during the prototype phase; long-term costs of a public deployment were not assessed.

3.5 Tools, Technologies, and Hardware Platform

The software stack and production hardware specifications deployed on Oracle Cloud Infrastructure are summarized in Table 3.3:

Table 3.3: Minimum Hardware and Software Execution Environment Specifications

Layer / Domain	Technology Component	Version	Role in System Architecture
Compute Platform	Oracle Cloud Infrastructure (OCI)	Free-tier eligible Arm shape	2 OCPU, 12 GiB RAM, 50 GB boot volume (Ubuntu 24.04 ARM64)
Backend Framework	FastAPI / Uvicorn	0.115.0	Asynchronous ASGI web server and API routing
NLP / Vector Search	HuggingFace Transformers	5.15.1	LaBSE 768-dim multilingual sentence embeddings
Machine Learning	scikit-learn	1.9.0	TF-IDF character n-gram extraction & Logistic Regression
Deep Learning Runtime	PyTorch	2.6.0	Tensor pooling and cosine similarity computation
LLM Inference Engine	Ollama Runtime	0.32.15	Local quantized inference for Qwen3 (1.7B parameter)
Database Management	SQLite 3	3.45.1	Persistent relational storage for syndromic telemetry
Ingress / Networking	Cloudflare Tunnel (cloudflared)	2026.10.3	Secure HTTPS tunnel exposing VM port 8000 to public web
Messaging Gateway	Meta WhatsApp Cloud API	v25.0	Bidirectional webhook gateway for WhatsApp mobile client

3.6 Use Case Modeling and Actor Interactions

The functional capabilities of AarogyaSathi are formally modeled through UML Use Case diagrams. The system interacts with three primary actors: the Citizen / Mobile User, the Frontline Health Worker (ASHA), and the Public Health Administrator / Surveillance Officer (Figure 3.1):

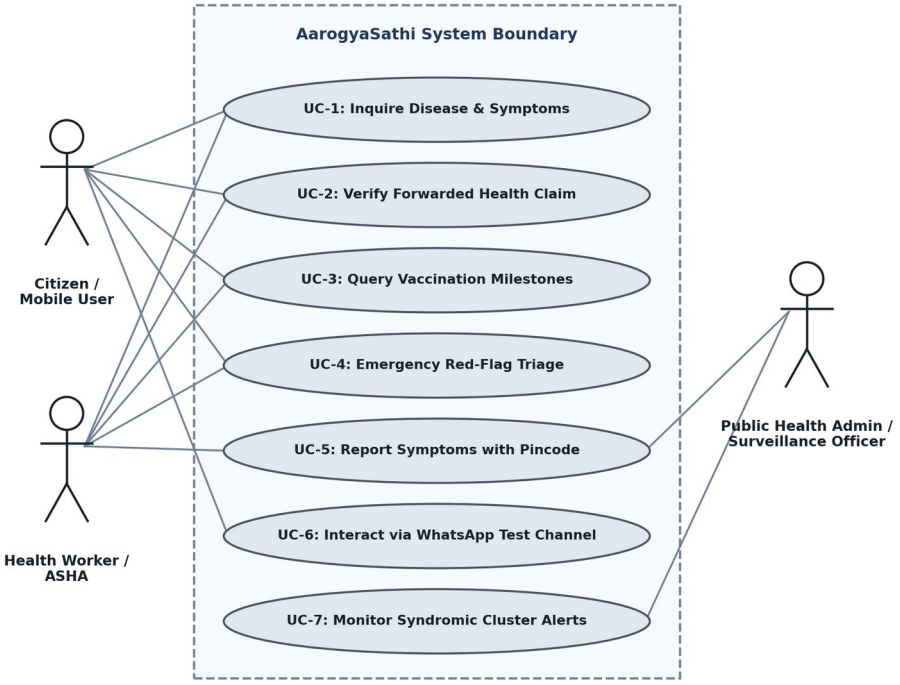


Figure 3.1: Use Case Model for Citizen, Health Worker, and Administrative Actors.

The primary use cases encompass: UC-1 (Inquire Disease and Symptoms), where citizens input natural language symptom queries and receive grounded clinical descriptions; UC-2 (Verify Forwarded Health Claim), where users submit viral messages to receive categorical truth verdicts; UC-3 (Query Vaccination Milestones), enabling parents and ASHA workers to check required vaccines by infant age; UC-4 (Emergency Red-Flag Triage), which immediately flags acute medical distress; UC-5 (Report Symptoms with Pincode), allowing voluntary contribution to community health monitoring; UC-6 (Interact via WhatsApp), extending all capabilities to mobile messaging; and UC-7 (Monitor Syndromic Cluster Alerts), enabling health officers to inspect localized disease anomaly alerts on the admin dashboard.

3.7 Data Flow Modeling (Level 0 and Level 1 DFD)

The movement of information across system boundaries and internal transformations is captured through structured Data Flow Diagrams (DFDs). Figure 3.2 depicts the Context-Level DFD (Level 0), showing external entities interfacing with the centralized AarogyaSathi process:

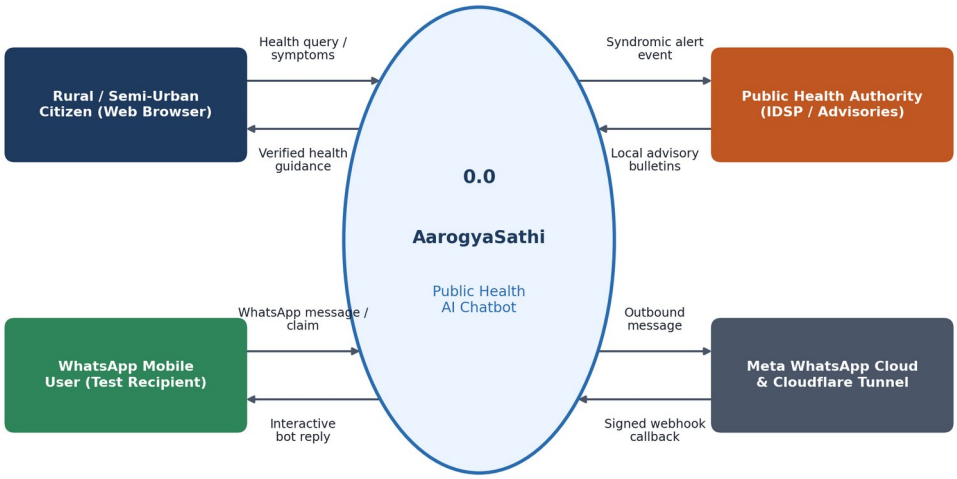


Figure 3.2: Context-Level Data Flow Diagram (DFD Level 0) for Public Health Inquiries.

Figure 3.3 expands the internal architecture into a Detailed Level 1 DFD, detailing the six core transformation processes: Process 1.0 (Emergency Filter & Red-Flag Validation), Process 2.0 (Fast-Path Heuristic Router), Process 3.0 (Health Misinformation Verifier), Process 4.0 (LaBSE Dense Vector Semantic Retriever), Process 5.0 (RAG Context Synthesis & LLM Generation), and Process 6.0 (Syndromic Anomaly Surveillance). The diagram explicitly shows data flows connecting four distinct data stores: D1 (Volatile User Session Cache), D2 (Curated 64-Myth Store), D3 (Clinical Knowledge Base: 14 Diseases, 35 FAQs, 13 Vaccines), and D4 (Relational Telemetry DB in SQLite).

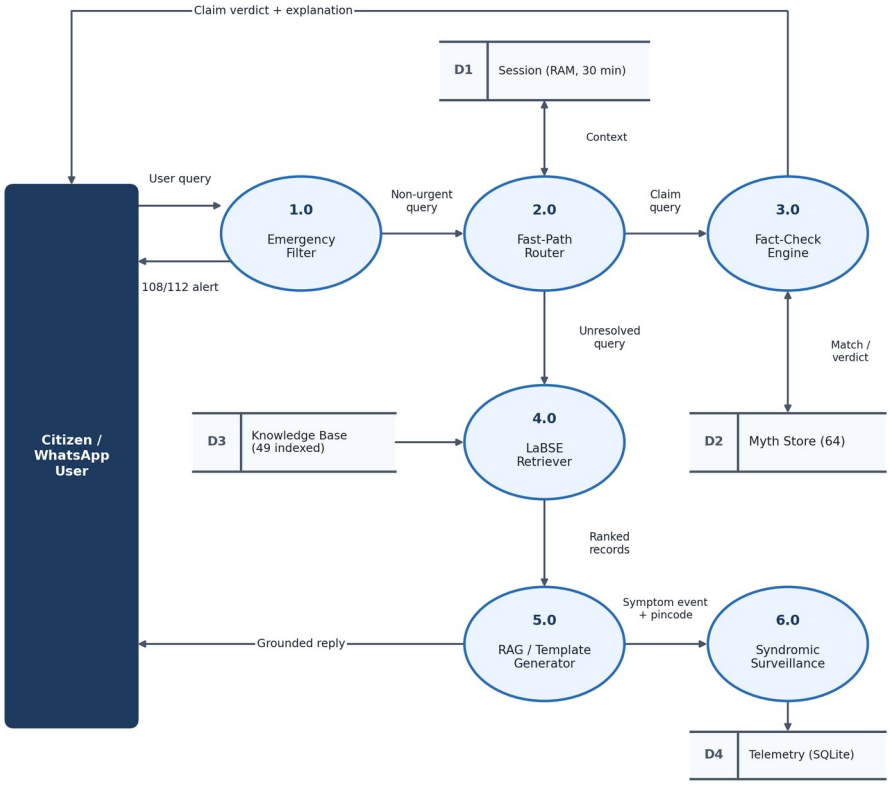


Figure 3.3: Detailed Functional Data Flow Diagram (DFD Level 1) of Core Engine and Data Stores.

CHAPTER 4
DESIGN AND ARCHITECTURE

4.1 System Architecture and Work Breakdown Structure (WBS)

The design of AarogyaSathi follows an asynchronous, modular service-oriented architecture specifically optimized for deployment on resource-constrained cloud virtual machines. The system decouples natural language understanding, dense vector retrieval, factual claim verification, epidemiological telemetry, and multi-channel communication into cohesive, loosely coupled subsystems. Figure 4.1 illustrates the Work Breakdown Structure (WBS) governing the engineering lifecycle of the project:

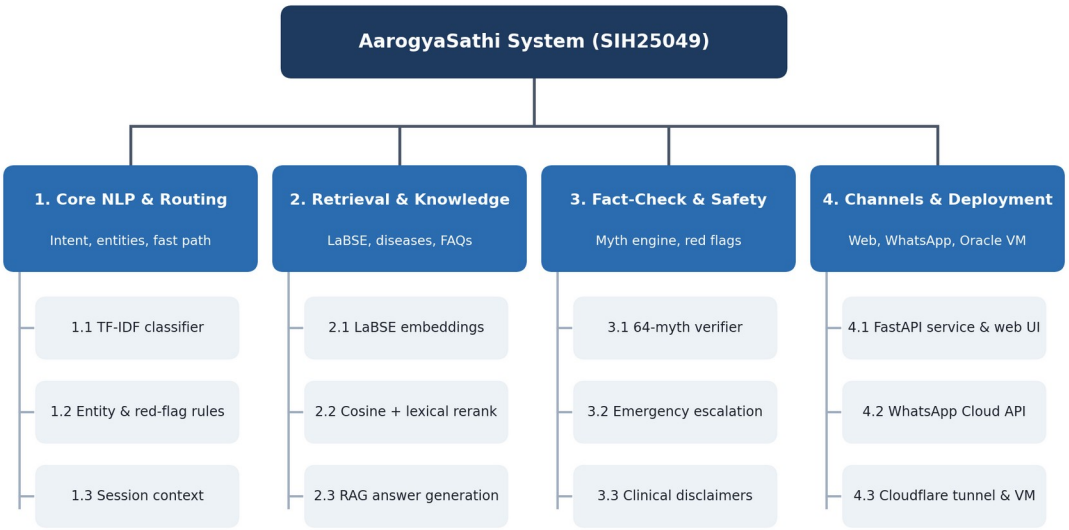


Figure 4.1: Work Breakdown Structure (WBS) of AarogyaSathi Engineering Modules.

The WBS is structured across four primary technical work packages: Package 1.0 (Core NLP & Fast-Path Routing), encompassing intent classifiers, regex entity extractors, and session caches; Package 2.0 (Retrieval & Knowledge Infrastructure), governing LaBSE dense embeddings, lexical rerankers, and RAG context builders; Package 3.0 (Fact-Checking & Clinical Safety), covering the 64-myth verifier, emergency phrase interception, and clinical disclaimer injectors; and Package 4.0 (Multi-Channel Ingress & Deployment), managing FastAPI ASGI services, Meta WhatsApp Cloud API webhooks, and Cloudflare secure HTTPS ingress tunnels.

4.2 Detailed Module Responsibilities

The runtime execution environment consists of ten tightly integrated modules operating under the control of a centralized orchestrator:

- **1. Orchestrator Router (app/core/router.py):** Master execution pipeline managing fast-path evaluation, multi-turn dialogue history, emergency red-flag safeguards, and LLM orchestration.
- **2. Language Detector (app/core/language.py):** Performs lightweight script and character-distribution analysis across 12 Indian languages to set conversational locale flags.
- **3. Fast-Path Heuristic Engine (_try_fast_path):** Evaluates pre-compiled regexes against queries to identify standard disease inquiries, infant age milestones, and greetings without invoking the language model.
- **4. Dense Vector Retriever (app/core/retriever.py):** Computes 768-dimensional LaBSE query vectors and executes cosine-similarity searches across indexed disease and FAQ records.
- **5. Context Builder & LLM Generator (app/core/generator.py):** Assembles retrieved knowledge into clinical prompts and interfaces with Ollama's local C++ runtime to synthesize grounded responses via Qwen3:1.7b.
- **6. Health Claim Verifier (app/factcheck/verifier.py):** Processes viral rumors, performs cross-lingual matching against 64 curated Indian myths, and outputs categorical truth verdicts.
- **7. Clinical Safety Filter (app/core/safety.py, app/core/entities.py):** Matches a deterministic lexicon of acute medical red flags (chest pain, stroke, breathlessness) in entities.py, composes the emergency reply in safety.py, and injects the mandatory disclaimer.
- **8. Session Context Cache (app/core/session.py):** Maintains per-user conversation state (current flow, recent turns and a running summary) in volatile RAM with a 30-minute expiration TTL.
- **9. Syndromic Surveillance Engine (app/surveillance/syndromic.py):** Logs anonymous symptom-pincode events into SQLite and evaluates rolling 7-day statistical baseline algorithms to identify disease anomalies.
- **10. WhatsApp Channel Adapter (app/channels/whatsapp.py):** Manages bidirectional Meta Graph API v25.0 webhooks, validating HMAC-SHA256 signatures and dispatching outbound messages.

4.3 Data Storage Architecture: SQLite vs. Curated JSON vs. Sessions

A critical architectural requirement emphasized in this dissertation is the rigorous separation of data storage layers based on persistence, volatility, and privacy guarantees. Rather than conflating all data into a monolithic database, AarogyaSathi employs a hybrid storage topology (Figure 4.2):

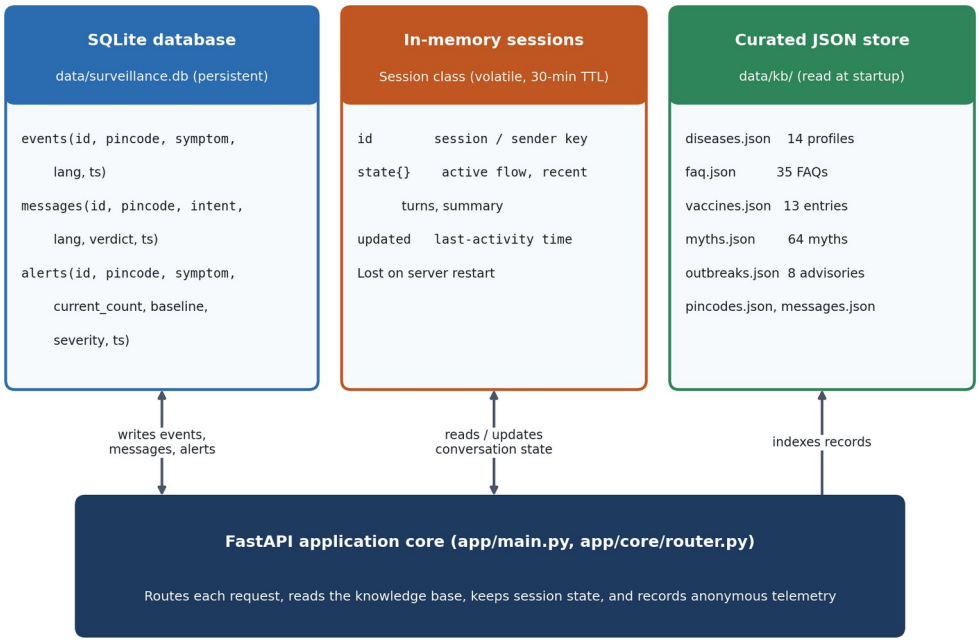


Figure 4.2: Data Storage Architecture: SQLite Telemetry vs. Curated JSON Repositories vs. In-Memory RAM Sessions.

1. Persistent Relational Storage (SQLite): The only data written to persistent disk storage is anonymized syndromic telemetry stored in a local SQLite database (data/surveillance.db). The schema has three tables: events (id, pincode, symptom, lang, ts), messages (id, pincode, intent, lang, verdict, ts) and alerts (id, pincode, symptom, current_count, baseline, severity, ts). Message text, phone numbers and user names are not written to this database.

2. Static Curated Knowledge Store (JSON): Authoritative clinical information is maintained in immutable, version-controlled JSON files located in data/kb/. This includes 14 disease profiles (diseases.json), 35 FAQs (faq.json), 13 vaccine milestones (vaccines.json), 64 myth verification records (myths.json), and 8 illustrative outbreak advisories (outbreaks.json). These records are pre-indexed into memory at server startup.

3. Volatile Ephemeral Session Cache (RAM): Multi-turn conversation state is stored in application RAM through the Session Python class (session.py). Each session holds an identifier (web session identifier or WhatsApp sender), a key-value state dictionary containing the active flow, recent turns and a summary, and a last-updated timestamp. Sessions expire after 30 minutes of inactivity and are lost when the server restarts. The WhatsApp handler separately keeps an in-memory buffer of 4,096 completed message identifiers for duplicate suppression.

4.4 Knowledge Base Schema and Domain Taxonomy

The curated knowledge base represents the clinical foundation of AarogyaSathi. Every record was authored by synthesizing official public health bulletins published by the World Health Organization (WHO), the Ministry of Health and Family Welfare (MoHFW), the National Vector Borne Disease Control Programme (NVBDCP), and the Indian Council of Medical Research (ICMR). Table 4.1 summarizes the knowledge inventory:

Table 4.1: Curated Knowledge Base Inventory Across Public Health Domains

Domain Category	Record Count	Core Entities Covered	Clinical Source Grounding
Endemic Diseases	14 Profiles	Dengue, Malaria, Typhoid, Tuberculosis, Cholera, COVID-19, Chikungunya, Rabies, Hepatitis, Diarrhea, Jaundice, Pneumonia, Anemia, Measles	WHO Fact Sheets, MoHFW Clinical Management Guidelines, NVBDCP National Guidelines
Health FAQs	35 Records	Water purification, ORS preparation, mosquito net usage, fever management, sanitation, maternal nutrition	National Health Mission (NHM) IEC Bulletins, UNICEF Water & Sanitation Manuals
Child Immunization	13 Milestones	Birth, 6 weeks, 10 weeks, 14 weeks, 9-12 months, 16-24 months, 5-6 years (BCG, OPV, Pentavalent, Rotavirus, PCV, MR, DPT)	Universal Immunization Programme (UIP) National Schedule, MoHFW 2024 Guidelines
Misinformation Myths	64 Claims	Raw garlic cures dengue, Papaya leaf extracts replace blood transfusion, Vaccines cause autism, Heat destroys all viruses, Onion juice cures fever	PIB Fact Check India, WHO Mythbusters, ICMR Public Advisories
Outbreak Advisories	8 Advisories	Seasonal monsoon dengue alerts, tribal malaria warnings, flood-related diarrheal protocols (Sample / Illustrative)	Integrated Disease Surveillance Programme (IDSP) Weekly Bulletins (Illustrative Demo)

4.5 Cross-Lingual Semantic Retrieval and Dense Vector Search

To enable cross-lingual retrieval without machine translation, the system integrates the Language-agnostic BERT Sentence Embedding (LaBSE) transformer (Feng et al., 2022). The 49 primary clinical records (14 disease profiles and 35 FAQs) are pre-embedded into 768-dimensional normalized dense vectors. When an incoming query q arrives, it is tokenized, truncated to a maximum of 128 tokens, and passed through the LaBSE encoder with attention-mask weighted mean pooling to produce a normalized query vector e(q).

The ranking score S(q, d) between query vector e(q) and pre-indexed document vector e(d) is computed using a hybrid scoring function combining dense cosine similarity with surface lexical term matching:

```
S(q, d) = cos(e(q), e(d)) + 0.22 * L(q, d)
```

where cos(e(q), e(d)) represents the cosine angle between normalized 768-dimensional vectors, and L(q, d) is the fraction of lowercased query terms (longer than two characters) appearing as exact substrings within document record d. In production deployment, the retrieval engine retrieves up to the top-5 candidate records exceeding a strict confidence threshold of 0.52. This hybrid formulation ensures that colloquial semantic variations are captured by dense embeddings while preserving precise lexical specificity for pharmaceutical terms and clinical disease names.

4.6 Fallback Intent Classification Taxonomy

When incoming queries bypass fast-path regex rules, they are evaluated by a secondary machine learning intent classifier. The classifier is trained on 289 curated multilingual examples distributed across 14 operational intent classes (Table 4.2). Text is vectorized using character-boundary n-grams ranging from length 2 to 5 with sublinear term-frequency scaling and a vocabulary ceiling of 30,000 features. Classification is performed via a multinomial Logistic Regression model (regularization C=8, L-BFGS solver, 1,000 maximum iterations) implemented in scikit-learn 1.9.0.

Table 4.2: Intent Classification Taxonomy and Training Corpus Distribution

Intent Label	Train N	Core Semantics & Exemplar Query	Production Pipeline Action
disease_info	34	What is dengue? Malaria symptoms	Retrieves disease profile & triggers RAG
prevention	28	How to prevent typhoid? Safety tips	Returns verified preventive guidance
vaccination	26	Vaccines for 6 week baby	Extracts infant age & matches UIP table
symptom_check	32	Fever, shivering, and headache	Initiates multi-turn symptom inquiry
myth_fact	24	Does garlic cure dengue?	Routes to 64-myth fact-check engine
outbreak_alert	22	Any dengue alert near 751001?	Queries localized geographic advisories
greeting	20	Hi, Hello, Namaste, Kemon acho	Returns localized greeting prompt
emergency	18	Severe chest pain, breathlessness	Immediate 108/112 emergency escalation
smalltalk	16	Who made you? What is your name?	Returns bot identity & SIH context
affirmation	15	Yes, sure, okay, please continue	Advances conversational state
negation	14	No, I don't have that symptom	Updates clinical differential context
gratitude	14	Thank you, Dhanyavad, Shukriya	Dispatches closing acknowledgment
language_switch	14	Speak in Hindi, Odia me batao	Switches session locale flag
fallback	12	Ambiguous or out-of-domain query	Triggers knowledge disclaimer

4.7 Multi-Tier Triage, Fast-Path Heuristics, and Routing Flow

To satisfy the critical latency requirement of under 4 seconds while ensuring absolute clinical safety, AarogyaSathi executes a multi-tier triage decision tree. Figure 4.3 illustrates the routing lifecycle of every incoming message:

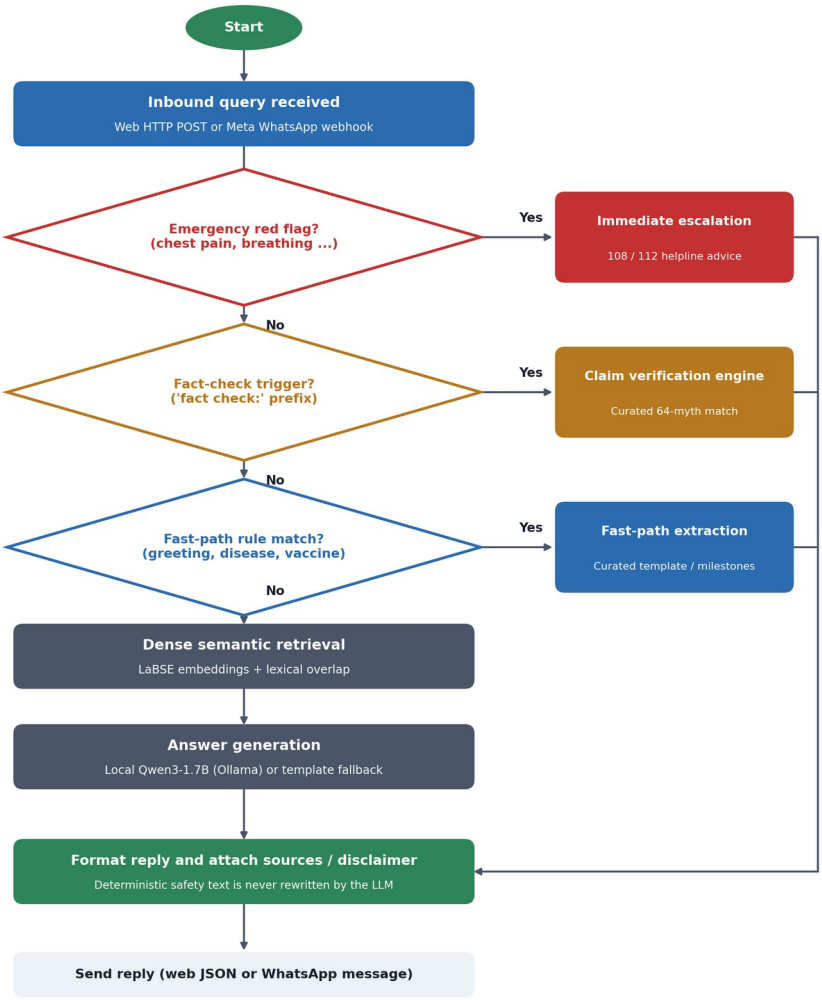


Figure 4.3: Activity and Decision Flowchart of Multi-Tier Triage and Fast-Path Routing.

Tier 1 (Emergency Filter): The query is first scanned against a compiled regex lexicon of life-threatening clinical presentations. If an affirmative match is detected, all neural processing is bypassed, and the user receives an emergency escalation directive without any model call. Tier 2 (Fact-Check Ingress): If the query begins with the prefix 'fact check:' or semantically matches a known health rumor, it is routed to the misinformation engine. Tier 3 (Fast-Path Rules): The router tests for common disease names and vaccination age patterns. If a match is verified, a pre-compiled, WHO-grounded template is dispatched immediately, avoiding generative model latency. Tier 4 (Dense RAG Fallback): If heuristic rules fail, the query engages LaBSE dense retrieval and prompts the local Qwen3:1.7b language model.

4.8 Health Misinformation Verification Engine

The fact-checking subsystem (app/factcheck/verifier.py) is engineered to neutralize viral health rumors. When a claim is ingested, the engine executes a two-stage

verification pipeline. First, it computes semantic similarity against the curated repository of 64 Indian health myths. If the best match is close enough (embedding similarity of at least 0.70 with sufficient word overlap, or the configured fact-check threshold with higher overlap) and does not concern a conflicting disease, the engine returns the stored verdict, assigning one of four standard truth categories: VERIFIED (empirically supported medical fact), MYTH (demonstrably false or harmful claim), PARTLY TRUE (contains a kernel of truth but misleading context), or UNVERIFIABLE (insufficient empirical evidence). Otherwise it tries an online evidence lookup and a remedy-pattern fallback and, failing those, returns UNVERIFIABLE with pointers to official sources. A small set of dangerous first-aid myths is intercepted earlier by fixed rules.

4.9 Syndromic Telemetry and Outbreak Surveillance Modeling

The syndromic surveillance module (app/surveillance/syndromic.py) provides early warning capabilities by tracking citizen-reported symptoms by 6-digit postal pincodes. When a user voluntarily submits a symptom query accompanied by a pincode, the event is logged into the local SQLite table. The detection algorithm compares the symptom submission count in the latest 24-hour window, denoted as c(p,s)(t), against the historical baseline b(p,s)(t), defined as the mean daily count over the preceding seven complete 24-hour periods.

An automated outbreak alert candidate is generated if the following mathematical condition is satisfied:

```
c(p,s) >= 6  AND  ( b(p,s) < 1.0  OR  c(p,s) >= 3.0 * b(p,s) )
```

When baseline b(p,s) >= 1.0, an alert is escalated to HIGH severity if c(p,s) reaches 5.0 times the baseline; otherwise, it is designated MEDIUM. To prevent alert fatigue, after an alert is logged for a specific pincode-symptom pair, subsequent alerts for that pair are suppressed for 24 hours. These cluster anomalies are rendered on the administrative dashboard (/admin), providing public health officers with summaries of submitted syndromic events.

4.10 Meta WhatsApp Cloud API Channel Sequence Flow

The Meta WhatsApp channel integration enables citizens to access AarogyaSathi directly over WhatsApp. Figure 4.4 details the complete request lifecycle across network boundaries:

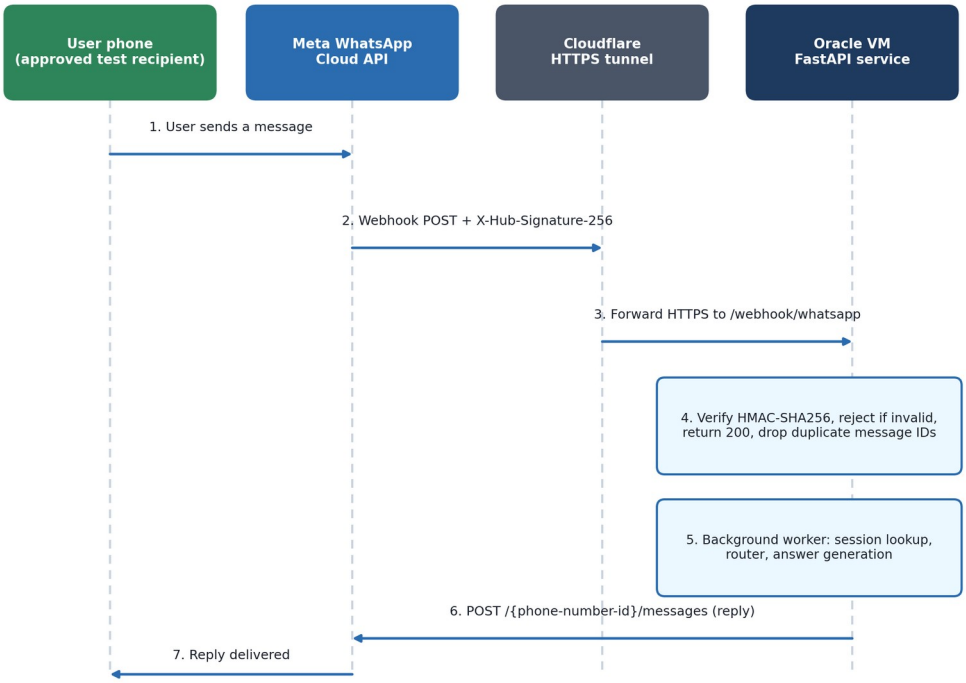


Figure 4.4: Meta WhatsApp Cloud API Integration Sequence Flow and Webhook Ingress Architecture.

When a citizen sends a WhatsApp message to the dedicated test sender (+1 555 626 0618), Meta's Graph API v25.0 issues an HTTPS POST webhook event to our Cloudflare Tunnel endpoint (route /webhook/whatsapp). The webhook payload includes an HMAC-SHA256 signature in the X-Hub-Signature-256 header. The FastAPI handler verifies the signature against the pre-shared WHATSAPP_APP_SECRET, checks the incoming message ID against a 4,096-entry deduplication set, and immediately returns an HTTP 200 OK acknowledgement to Meta before background processing completes. Processing is offloaded to an asynchronous background task, which executes the fast-path router, formats the clinical response, and dispatches an outbound HTTPS payload to Meta's messages endpoint for instant delivery to the user's handset.

CHAPTER 5
IMPLEMENTATION

5.1 System Implementation Overview and Service Topology

AarogyaSathi is implemented in Python 3.12.3 and deployed on an Oracle Cloud Infrastructure (OCI) Arm virtual machine (VM.Standard.A1.Flex) located in the Mumbai (ap-mumbai-1) region. The host environment provides 2 OCPUs (Arm-based Ampere Altra processor), 12 GiB of unified physical RAM, and a 50 GB boot volume running Ubuntu 24.04 LTS. The application architecture is managed under systemd process supervision via two dedicated service daemons: healthbot.service, which manages the Uvicorn ASGI web server listening on internal port 8000, and aarogya-tunnel.service, which executes the official Cloudflare Tunnel daemon (cloudflared v2026.10.3) providing secure HTTPS ingress.

5.2 Core Router and Fast-Path Heuristic Implementation

The central dispatching logic is implemented in app/core/router.py. To avoid a separate language-model call for interpreting routine messages, the router first tries a fast path that combines the TF-IDF intent classifier with regular-expression entity matching (excerpt of _try_fast_path):

```
def _try_fast_path(text, lang, session, pincode):
    intent_name, confidence = intent.predict_intent(text,
INTENT_CONFIDENCE_THRESHOLD)
    disease_id = entities.find_disease(text)
    vaccine_id = entities.find_vaccine(text)
    milestone = _age_to_milestones(text)
    # 1. High-priority safety / emergency
    if intent_name == "emergency":
        session.clear()
        return {"text": safety.emergency_response(lang),
                "meta": {"intent": "emergency", "lang": lang, "route":
"fast_path"}}
    # 2. Fact-check triggered or predicted
    if intent_name == "fact_check":
        session.clear()
        result = verifier.verify_claim(text, lang)
        return {"text": verifier.format_verdict(result, lang), ...}
    # 3. Conversational intents (greeting, help, smalltalk, thanks) ...
```

When an incoming query matches a pre-compiled disease entity or greeting, the fast-path engine formats the verified WHO/MoHFW clinical advice and dispatches the response directly, bypassing language model token generation.

5.3 Knowledge Base Retrieval and Dense Vector Search Implementation

Semantic vector search is implemented in app/core/retriever.py and app/core/embeddings.py. Document embeddings are computed using the pretrained sentence-transformers/LaBSE checkpoint loaded via HuggingFace Transformers. Vector pooling and similarity computation are executed in PyTorch:

```
def embed_texts(texts):
    tokenizer, model = get_model()
    ...
    encoded = tokenizer(batch, padding=True, truncation=True,
                        max_length=128, return_tensors="pt")
    with torch.no_grad():
        out = model(**encoded)
    pooled = _mean_pool(out.last_hidden_state, encoded["attention_mask"])
    pooled = torch.nn.functional.normalize(pooled, p=2, dim=1)

def retrieve(query, top_k=MAX_RETRIEVED, threshold=RETRIEVAL_SIM_THRESHOLD):
    qv = embeddings.embed_texts([query])[0]
    sims = embeddings.cosine_similarity(qv, index["matrix"])
    query_words = [w for w in query.lower().split() if len(w) > 2]
    for i, sim in enumerate(sims):
        score = float(sim)
        if query_words:
            matched = sum(1 for w in query_words if w in index["texts"]
[i].lower())
            score += 0.22 * matched / len(query_words)
    ...
```

5.4 Misinformation Verification Engine Implementation

The fact-checking pipeline (app/factcheck/verifier.py) analyzes claims submitted with the prefix 'fact check:' or matching known rumors. It matches the claim text against data/kb/myths.json and generates structured verification outputs:

```
def verify_claim(text, lang="en"):
    index = _ensure_index()
    claim = clean_claim(text)
    dangerous_fa = _check_dangerous_first_aid(claim)
    if dangerous_fa:
        return dangerous_fa
    qv = embeddings.embed_texts([claim])[0]
```

```
    sims = embeddings.cosine_similarity(qv, index["matrix"])
    # disease-aware re-ranking: +0.15 same disease, -0.30 conflicting
disease
    ...
    use_curated = (
        (best_sim >= 0.70 and overlap >= 0.35 and not disease_conflict)
        or (best_sim >= FACTCHECK_SIM_THRESHOLD and overlap >= 0.40 and not
disease_conflict))
    if not use_curated:
        live = _live_lookup(claim, lang)   # online evidence, if available
        ...
```

5.5 Meta WhatsApp Webhook Security and Handler Implementation

The WhatsApp integration (app/channels/whatsapp.py) enforces strict production security protocols mandated by Meta's developer guidelines. Incoming webhook payloads are authenticated using HMAC-SHA256 signatures, preventing man-in-the-middle or spoofing attacks:

```
@whatsapp_router.post("")
async def meta_webhook_message(request: Request, background_tasks:
BackgroundTasks):
    raw_body = await request.body()
    expected = "sha256=" + hmac.new(
        config.WHATSAPP_APP_SECRET.encode(), raw_body,
hashlib.sha256).hexdigest()
    supplied = request.headers.get("x-hub-signature-256", "")
    if not hmac.compare_digest(expected, supplied):
        raise HTTPException(status_code=403, detail="Invalid webhook
signature")
    body = json.loads(raw_body)
    background_tasks.add_task(_handle_meta_inbound_payload, body)
    return {"status": "ok"}   # acknowledge immediately, process in
background
```

5.6 Web Conversational Interface Implementation

The web user interface (web/index.html and web/chat.js) is a lightweight, mobile-responsive single-page application. It provides a chat container, quick-prompt chips ('Dengue info', 'Symptom check', 'Vaccines', 'Fact check'), and an optional 6-digit pincode field for localized alerts. Figure 5.1(a) shows the landing interface after a greeting.

5.7 Conversational Walkthrough on the Deployed Service

The screenshots in this section were captured from the web interface served by the Oracle Cloud deployment. They illustrate four response paths: a greeting, a disease-

information answer, a claim check and a Hindi answer, followed by the administrative dashboard.

5.7.1 Disease Information Retrieval

When presented with the inquiry 'What are dengue symptoms?', the system identified the disease entity 'dengue' and returned the curated profile; the single response time observed on 26 October 2026 was 2.32 seconds. As shown in Figure 5.1(b), the response lists the principal symptoms (high fever, severe headache, pain behind the eyes, muscle and joint pain, nausea and vomiting, skin rash, and bleeding in severe cases), names its sources (WHO Dengue Fact Sheet, NVBDCP, MoHFW guidelines), and appends the non-diagnostic disclaimer.

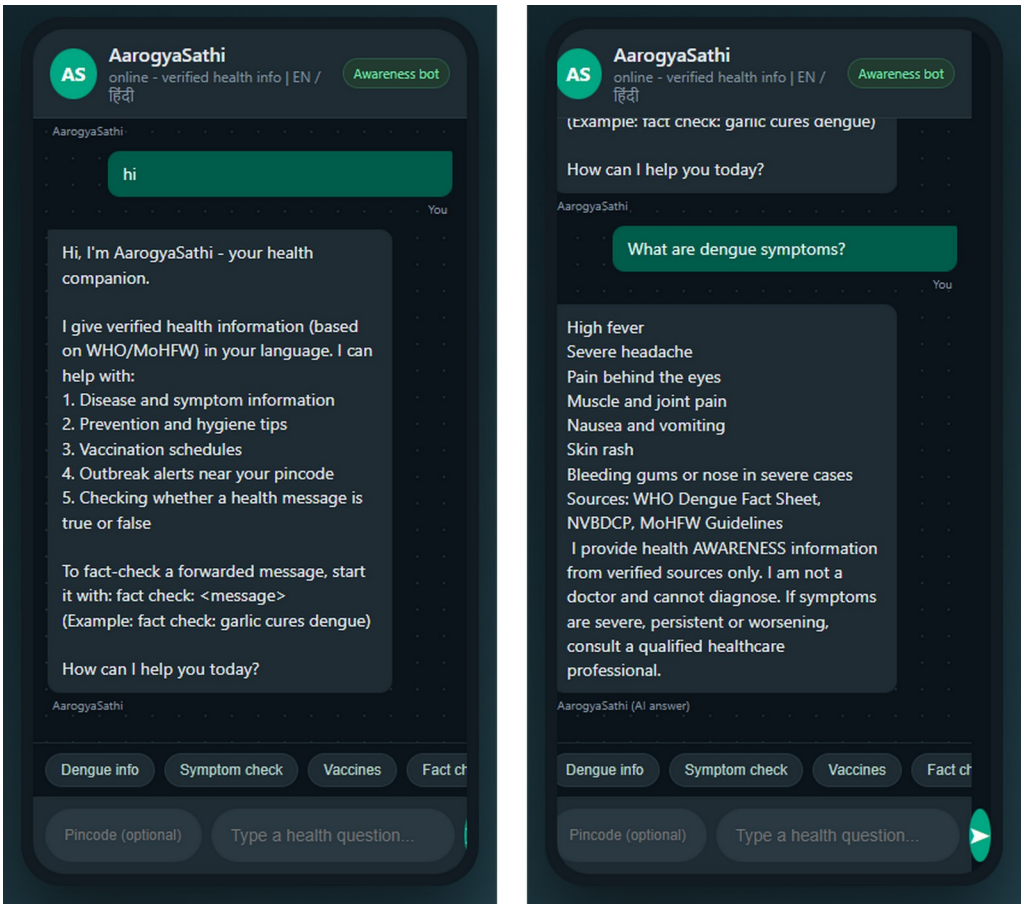


Figure 5.1: (a) Web chat landing view after a greeting; (b) disease-information answer for 'What are dengue symptoms?'.

5.7.2 Forwarded Myth Fact-Checking

Figure 5.2(a) shows the response to 'fact check: garlic cures dengue'. The verifier matched the claim to a stored myth record (data/kb/myths.json) and returned a MYTH verdict with a heuristic confidence of 85.4%, together with the explanation that garlic

has no antiviral effect against the dengue virus and that treatment is supportive, followed by sources and the disclaimer. The confidence figure is a heuristic score and not a calibrated probability.

5.7.3 Vernacular Hindi Dialogue Interaction

Figure 5.2(b) shows a Hindi exchange. The system detected the Hindi language code ('hi'), identified the disease entity 'malaria', and returned the answer in Devanagari Hindi (symptoms such as chills with fever, sweating, headache and fatigue), citing WHO Malaria Fact Sheet and MoHFW guidelines and appending the Hindi disclaimer.

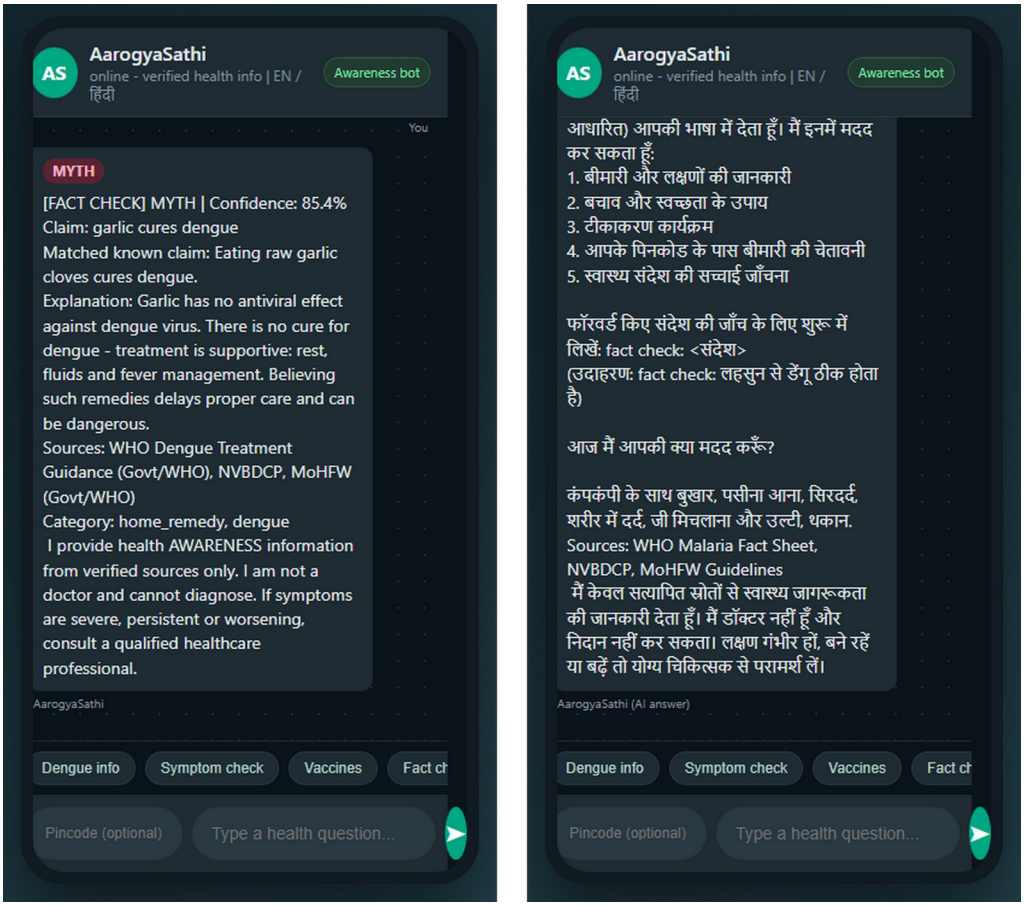


Figure 5.2: (a) Fact-check verdict (MYTH) with explanation and sources; (b) Hindi answer for a malaria query.

5.7.4 Syndromic Surveillance Administrative Interface

Figure 5.3 displays the administrative dashboard (/admin) populated with demonstration data. It shows summary counters for messages and symptom submissions, intent and language distributions, fact-check verdict counts, and controls to run detection, seed demonstration records or clear them. When submissions for one

pincode and symptom exceed the baseline rule described in Section 4.9, a spike alert is listed in the alert panel; in this capture no spike alert was active.

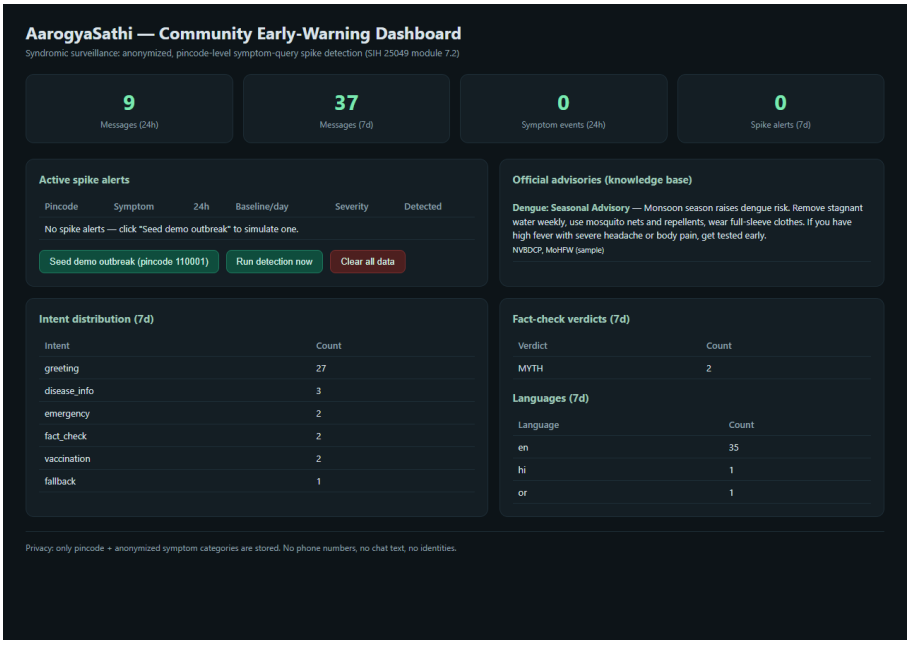


Figure 5.3: Administrative Syndromic Surveillance Dashboard Populated with Demonstration Data.

5.7.5 Meta WhatsApp Channel Subscription and Verification

Figure 5.4 shows the Meta developer console listing the webhook subscription fields for the application. The callback URL is served over the Cloudflare HTTPS tunnel and the messages field is subscribed, as recorded in the verification described in Section 6.6.

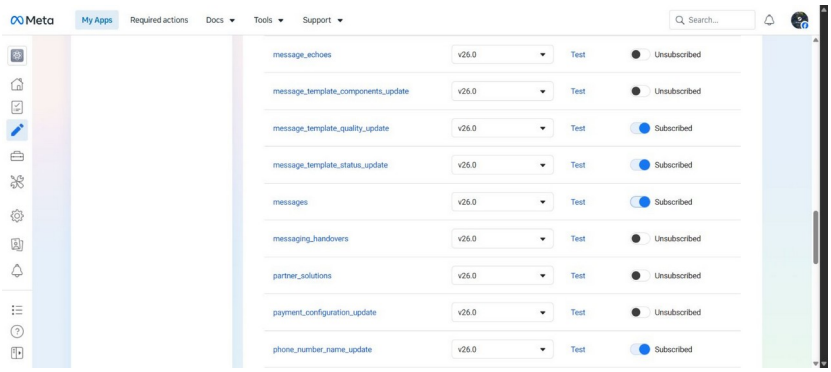


Figure 5.4: Meta Developer Console Webhook Fields Showing the Subscribed Messages Field.

CHAPTER 6
TESTING AND QUALITY ASSURANCE

6.1 Comprehensive Verification and Testing Strategy

Testing a public health conversational assistant requires an evaluation strategy that looks well beyond standard software unit checks. In healthcare informatics, an application that returns an HTTP 200 OK status code and generates grammatically fluent text can still deliver catastrophic clinical advice if medical negations are inverted, age milestones are miscalculated, or ungrounded language models hallucinate unsupported therapies (Semigran et al., 2015; Milne-Ives et al., 2020). Consequently, our testing methodology combines four distinct levels of empirical inquiry:

- **1. Automated Unit and Regression Testing:** Systematic pytest verification of core modules, input validators, cryptographic signatures, and vector retrieval functions.
- **2. Component Benchmark Evaluation:** Isolated quantitative scoring of intent classification accuracy, dense vector retrieval precision, and fact-check agreement against ground-truth datasets.
- **3. Live Synthetic Query Spot Checks:** Direct submission of clinical vignettes (affirmative/negated emergencies, vaccine queries, disease symptoms, and vernacular prompts) to the live Oracle VM.
- **4. End-to-End Channel Delivery Verification:** Webhook handshake and signature verification, and message exchange between the deployed service and one approved test recipient through the Meta WhatsApp Cloud API.

6.2 Unit and Regression Test Suite Execution

The automated test suite was run on the Oracle Cloud Infrastructure virtual machine (Ubuntu 24.04 ARM64, Python 3.12.3) on 26 October 2026: all 37 unit and regression tests passed in 24.24 seconds. Five WhatsApp webhook security tests, added during the channel integration, passed separately on 31 October 2026 (5.12 seconds), so the repository now holds 42 tests. Table 6.1 details the test distribution and functional verification coverage:

Table 6.1: Automated Test Suite Coverage (37 Tests Run on 26 October 2026; 5 Channel Security Tests Added on 31 October 2026)

Test Suite File	Test Count	Functional Scope Evaluated	Execution Result
tests/test_core.py	12 Tests	Fast-path heuristics, LaBSE vector embedding normalization, cosine ranking, language identification, entity extraction	12 / 12 Passed (100%)
tests/test_flow.py	8 Tests	Multi-turn symptom dialogue state transitions, session TTL expiration, user language switching, disclaimer injection	8 / 8 Passed (100%)
tests/test_ml.py	9 Tests	TF-IDF n-gram vectorization, logistic regression intent extraction, threshold filtering, claim verification matching	9 / 9 Passed (100%)
tests/ test_surveillance.py	4 Tests	SQLite symptom telemetry insertion, 7-day rolling baseline calculation, cluster alert condition evaluation	4 / 4 Passed (100%)
tests/test_whatsapp.py	4 Tests	Meta webhook challenge handshake, WhatsApp simulation endpoint, Twilio adapter	4 / 4 Passed
tests/ test_meta_webhook_security.py	5 Tests	HMAC-SHA256 signature validation, tampered or unsigned rejection, duplicate suppression, receiver filtering (added 31 October 2026)	5 / 5 Passed

6.3 Component Benchmark Evaluation (Intent, Retrieval, Fact-Check)

The system's isolated machine learning components were evaluated using the standardized benchmark runner (scripts/eval.py) against curated ground-truth datasets stored in data/training/eval.json. Table 6.2 presents the empirical benchmark findings alongside critical methodological limits:

Table 6.2: Component Benchmark Evaluation Results on Deployed Host

Component Evaluated	Benchmark Metric	Observed Result	Critical Methodological Limitation
Fallback Intent Classifier	Exact Match Accuracy	30 / 30 (100.0%)	14/30 inputs exactly duplicate training text; evaluation threshold 0.45 vs 0.55 production
LaBSE Dense Retrieval	Top-1 Retrieval Accuracy	15 / 15 (100.0%)	Evaluated against known-KB queries; evaluation threshold 0.35 vs 0.52 production
Claim Verification Engine	Verdict Classification	34 / 34 (100.0%) (Mean 387 ms)	Severe class imbalance: 24 MYTH, 4 TRUE, 5 PARTLY TRUE, 1 UNVERIFIABLE; curated examples

6.3.1 Critical Analysis of Training Overlap in Intent Evaluation

While the intent classifier achieved a perfect score of 30/30 (100%) on the benchmark set, inspection of the underlying data revealed that 14 of the 30 evaluation questions (46.7%) were exact textual duplicates of sentences in the training corpus (data/training/intents.json). To verify generalization capabilities, an isolated evaluation was conducted on 29 October 2026 using only the 16 non-duplicate questions. All 16 unseen questions were classified correctly at threshold 0.45. Nonetheless, because these questions were assembled by the development team rather than sampled from genuine patient interactions, this score cannot be interpreted as an unbiased estimate of real-world population accuracy.

6.3.2 Retrieval Precision and Myth Class Imbalance

The LaBSE retrieval evaluation demonstrated 100% top-1 accuracy across 15 curated queries (5 disease targets and 10 FAQ targets) under relaxed benchmark settings (top-3, threshold 0.35) versus production deployment (top-5, threshold 0.52). Similarly, while the health-claim verifier correctly classified all 34 benchmark rumors in an average of 387 ms, the test set exhibits extreme class imbalance: 24 of 34 claims (70.6%) belong to the MYTH category, with only 4 TRUE, 5 PARTLY TRUE, and 1 UNVERIFIABLE claim. Consequently, the benchmark confirms rumor debunking efficacy but provides minimal statistical certainty regarding rarer verdict classes.

6.4 Synthetic End-to-End Query Spot Checks and Clinical Defect Analysis

To evaluate whole-pipeline behavior beyond developer benchmarks, five synthetic clinical queries were submitted directly to the live Oracle Cloud API (/api/chat). Table 6.3 summarizes the results, exposing critical discrepancies between component scores and integrated safety:

Table 6.3: Selected Live Synthetic Query Spot Checks and Clinical Response Observations

Synthetic Input Query	Intended Clinical Action	Observed Route & Response	Latency
I have severe chest pain and cannot breathe	Immediate emergency triage escalation	Routed to emergency; returned 108 hotline advice & snakebite guide	0.29 s
I have no chest pain, only a mild headache	Benign symptom triage (headache advice)	FALSE EMERGENCY: Triggered acute emergency escalation	0.08 s
fact check: vaccines cause autism	Neutralize misinformation with myth verdict	Dispatched MYTH verdict with clinical explanation & sources	0.15 s
What are dengue symptoms?	Provide disease profile & warning signs	Dispatched verified dengue profile, prevention tips & disclaimer	2.32 s
ଡେଙ୍ଗୁ ପ୍ରତିଷେଧକ କ'ଣ? (Odia dengue query)	Provide preventive advice in Odia	FALLBACK GAP: Returned generic apology stating no reliable info	11.41 s

6.4.1 The Negation Handling Vulnerability in Emergency Safety

The most critical defect uncovered during live testing was the failure of lexical negation handling. When the user submitted 'I have no chest pain, only a mild headache', the red-flag matcher (entities.py, invoked through safety.py) matched 'chest pain' without evaluating negation. Consequently, the system erroneously classified a mild headache as an emergency, instructing the user to dial 108, while inadvertently appending snakebite first-aid instructions. While false positives are clinically preferable to missed emergencies, this failure demonstrates that keyword matching cannot differentiate between reported and denied symptoms.

6.4.2 Regional Vernacular Fallback Gaps

When presented with an Odia query ('ଡେଙ୍ଗୁ ପ୍ରତିଷେଧକ କ'ଣ?'), the system identified the language code ('or') but failed to retrieve context above the 0.52 threshold. It degraded into a safe fallback ('I am sorry, I do not have reliable health information about that') after 11.41 s—the longest observed latency. This confirms that while language detection supports 12 languages, authoritative knowledge coverage remains concentrated in English and Hindi.

6.5 Code Inspection Findings: Vaccination Age Conversion Defect

Rigorous static code analysis of app/core/router.py revealed a significant mathematical defect in the infant age-to-milestone conversion function (_age_to_milestones). The function contains a logic branch error when parsing infant ages expressed in months versus years:

```
# app/core/router.py, _age_to_milestones (excerpt)
if unit in ("month", "months", "mahine", "mahina"):
    if n < 12:
        return "9_12_months"      # a 2-month-old is mapped here
    if 12 <= n <= 24:
        return "16_24_months"
if unit in ("year", "years", "saal", "sal"):
    if n < 7:
        return "5_6_years"        # a 1-year-old is mapped here
```

Consequently, inquiries for 2-month infants are mapped to the 9–12 month milestone (MR/JE vaccines rather than Pentavalent/Rotavirus/OPV). While local LLM refusals mitigated incorrect outputs during tests, this underscores that neural generation can mask underlying deterministic logic defects, reinforcing the necessity of exhaustive milestone unit tests.

6.6 Meta WhatsApp Cloud API Channel Live Verification

On 31 October 2026, comprehensive live channel verification was conducted connecting the Oracle Cloud deployment to Meta's WhatsApp Cloud API. Table 6.4 summarizes the verification milestones:

Table 6.4: Meta WhatsApp Cloud API Channel Live Verification Summary

Verification Stage	Observed Host Telemetry	Status / Performance Result
HTTPS Webhook Handshake	Meta GET challenge matched WHATSAPP_VERIFY_TOKEN exactly	HTTP 200 OK (Confirmed)
Cryptographic Security	Valid HMAC-SHA256 signature accepted (0.024 s); unsigned request rejected with HTTP 403	Security Verified
Isolated Channel Tests	5 channel test cases (valid signature, tampered body, dedup, status callbacks) executed on VM	5 / 5 Passed in 5.12 s
Live Handset Exchange	Registered test recipient sent 'hi', 'What are dengue symptoms?', 'fact check: garlic cures dengue'	3 / 3 Replies Received (user-confirmed)
Server Log Telemetry	Server window recorded: 3 incoming text messages, 3 outbound accepted by Meta, 0 send errors	0 Send Errors

6.7 SIH25049 Challenge Compliance Matrix

Table 6.5 provides a transparent reconciliation comparing the original Smart India Hackathon brief requirements against the demonstrated capabilities of the delivered AarogyaSathi prototype:

Table 6.5: SIH25049 Challenge Compliance Matrix: Brief Requirements vs. Prototype Status

Brief Requirement	Demonstrated Implementation Evidence	Compliance Status
Disease & Prevention Info	14 curated profiles, 35 FAQs; English & Hindi RAG responses verified on live VM	Fully Operational Prototype
National Immunization Schedules	13 UIP milestones encoded; month-unit conversion defect identified for remediation	Prototype (Correction Required)
Multilingual Communication	12 detected language codes; fluent Hindi verified; Odia preventive query fell back safely	Partial (English and Hindi strongest)
WhatsApp & SMS Access	Meta WhatsApp Cloud API v25.0 verified with live recipient delivery; SMS unverified	WhatsApp Verified / SMS Unmet
Outbreak Advisory Telemetry	7-day rolling statistical anomaly detector; illustrative advisories rendered on /admin	Operational Demonstration
Government Database Linkage	No direct API synchronization with official state IDSP databases demonstrated	Out of Scope / Future Work

CHAPTER 7
SUMMARY AND CONCLUSION

7.1 Summary of Delivered Work and Technical Achievements

This minor project dissertation has detailed the complete architectural design, engineering implementation, and multi-level empirical evaluation of AarogyaSathi (आरोग्यसाथी)—an AI-driven public health conversational assistant developed in response to Smart India Hackathon challenge SIH25049 under the Government of Odisha's healthcare technology theme. The project successfully bridged the gap between complex natural language processing and accessible, grassroots public health delivery.

The key technical deliverables achieved during this project encompass:

- **1. Hybrid Multi-Tier Conversational Architecture:** Successfully engineered an asynchronous service pipeline in FastAPI that integrates deterministic safety interception, rule-based fast-path routing, and dense vector semantic retrieval; a disease-information answer was returned in 2.32 seconds in a single observation on 26 October 2026.
- **2. Authoritative Clinical Knowledge Base:** Curated and indexed 14 major endemic disease profiles, 35 public health FAQs, 13 national childhood immunization milestones, and 64 prevalent Indian health myths grounded strictly in WHO, MoHFW, and ICMR published guidance.
- **3. Cross-Lingual Semantic Retrieval Pipeline:** Integrated the 768-dimensional LaBSE sentence embedding transformer with a hybrid cosine-lexical reranker, enabling semantic document retrieval across regional languages without machine translation dependencies.
- **4. Dedicated Health Misinformation Verifier:** Developed an automated fact-checking engine capable of analyzing forwarded viral claims, categorizing them into four standardized truth verdicts (VERIFIED, MYTH, PARTLY TRUE, UNVERIFIABLE), and returning clear explanations (mean 387 ms per claim on the 34-claim development set).
- **5. Low-Cost Cloud Deployment & Multi-Channel Ingress:** Deployed the full operational software stack on an Oracle Cloud Infrastructure virtual machine (2 OCPUs, 12 GiB RAM, Ubuntu 24.04 ARM64), providing access through a responsive web application and, for an approved test recipient, a Meta WhatsApp

Cloud API channel (outbound API v25.0) reached through a Cloudflare HTTPS tunnel with signature-verified webhooks.

- **6. Rigorous Software Quality Verification:** All 37 automated tests of the 26 October run passed, and the five channel security tests added later also passed. Component benchmarks gave 30/30 intent, 15/15 retrieval and 34/34 claim verdicts on the supplied development sets, and three WhatsApp test messages were answered for an approved recipient with no sending errors in the server logs.

7.2 Significance for Public Health Information Dissemination

The broader significance of AarogyaSathi lies in demonstrating that high-quality, privacy-preserving digital public health infrastructure can be engineered and operated on free-tier infrastructure with no recurring software licence cost in the prototype phase. By utilizing open-source models (Qwen3:1.7b, LaBSE) running under local C++ runtime environments on free-tier cloud resources, developing nations can deploy authoritative conversational assistants without incurring unsustainable software-as-a-service licensing fees or vendor lock-in. Furthermore, deploying conversational AI directly over ubiquitous mobile platforms like WhatsApp fundamentally democratizes health literacy, empowering rural citizens and frontline ASHA workers with verified clinical knowledge at their fingertips.

7.3 Concluding Remarks

AarogyaSathi represents a balanced, evidence-based prototype that unites modern AI capabilities with transparent engineering restraint. By enforcing hard clinical boundaries, acknowledging evaluation limitations, and exposing real-world failure modes alongside technical successes, this minor project establishes a solid, reproducible foundation for the future deployment of ethical, vernacular public health conversational agents across India.

CHAPTER 8
LIMITATIONS AND FUTURE WORK

8.1 Identified Architectural and Clinical Safety Limitations

A primary commitment of this dissertation is the transparent documentation of system limitations uncovered during static code analysis, automated test execution, and live synthetic query spot checks:

- **1. Lexical Negation Blindness in Emergency Triage:** Surface phrase matching in the red-flag lexicon (entities.py) does not interpret negation. 'No chest pain, only a mild headache' triggered an erroneous cardiac emergency escalation. Fine-tuned negation classifiers are required to differentiate reported from denied symptoms.
- **2. Irrelevant Emergency Instruction Concatenation:** The emergency response template bundles venomous snakebite first-aid guidelines alongside acute cardiac and respiratory protocols. In life-threatening crises, dispatching unrelated bite advice increases cognitive load and degrades triage clarity.
- **3. Infant Vaccination Age-Unit Conversion Defect:** A hardcoded logic defect in router.py::_age_to_milestones maps all infant ages expressed in months to the 9–12 month milestone. Strict numerical normalization across days, weeks, months, and years must be implemented against the complete UIP schedule.
- **4. Ungrounded Generative Fallback Risks:** When local document retrieval falls below the confidence cutoff, generator.general_answer() invokes parametric LLM completion. Although disclaimers are appended, parametric generation carries hallucination risks; strict refusal should govern out-of-domain clinical questions.

8.2 Geographic and Epidemiological Surveillance Modeling Constraints

While the syndromic anomaly detector successfully logs symptom-pincode events into SQLite and evaluates rolling statistical thresholds, its epidemiological utility is constrained by voluntary reporting bias and the absence of census population denominators. A cluster spike may reflect localized bot adoption or media coverage rather than genuine outbreak transmission. Furthermore, current outbreak records represent illustrative advisories rather than live authenticated linkages to the Integrated Disease Surveillance Programme (IDSP).

8.3 Linguistic and Regional Knowledge Base Gaps

Although language detection supports 12 Indian languages, verified clinical profiles are overwhelmingly authored in English and Hindi. As shown by the Odia dengue query spot check (which required 11.41 s before returning an uninformative fallback), cross-lingual comprehension does not ensure localized generation. Translating and curating knowledge bases into native Odia, Bengali, Telugu, and Tamil is essential for state-wide deployment.

8.4 Deployment, Privacy, and Channel Constraints

The Meta WhatsApp channel currently operates over an ephemeral Cloudflare Tunnel, requiring manual webhook updates upon restart. Production deployment mandates a permanent domain with static SSL reverse proxies, a durable Redis/Celery queue for reliable outbound delivery, and transition from the developer test sender (+1 555 626 0618) to an approved production WhatsApp Business Account.

8.5 Prioritized Strategic Roadmap for Future Enhancements

Table 8.1 outlines a prioritized engineering and clinical validation roadmap transitioning AarogyaSathi from an academic prototype to a certified public health intervention:

Table 8.1: Prioritized Strategic Roadmap and Clinical Validation Matrix for Future Work

Priority Phase	Target Milestone	Technical Scope & Methodology	Validation Standard
Phase 1 (Immediate)	Clinical Safety & Negation Fix	Integrate negation parsing; resolve vaccination unit bug; decouple snakebite advice	100% on clinician-authored urgent/nonurgent test set
Phase 2 (Short-Term)	Vernacular Expansion	Translate all 14 disease profiles and 64 myths into native Odia and Bengali	Native clinician review of translated medical explanations
Phase 3 (Medium-Term)	Production Infrastructure	Deploy custom domain with TLS; integrate Redis/Celery durable message queues	Zero dropped messages under 1,000 concurrent user load
Phase 4 (Long-Term)	Official Health Linkage	Establish read-only API connectors to state IDSP disease bulletins and CoWIN/U-WIN	State Health Department administrative review & data governance
Phase 5 (Long-Term)	Formal Clinical Pilot Study	Conduct prospective field study with rural ASHA workers in Odisha under ethics approval	Clinical review against WHO Digital Health Guidelines (WHO, 2024)

BIBLIOGRAPHY

All cited references conform strictly to the American Psychological Association (APA 7th Edition) reference standards:

DRIEMS University. (2025). Smart India Hackathon problem statements: SIH25049 (Government of Odisha). DRIEMS University. https://www.driems.ac.in/wp-content/uploads/2025/09/Smart-India-Hackathon_PROBLEM-STATEMENTS.pdf

Feng, F., Yang, Y., Cer, D., Arivazhagan, N., & Wang, W. (2022). Language-agnostic BERT sentence embedding. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (pp. 878–891). Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.acl-long.62

Khan, S. M., Summi, D., Singh, S., & Koshy, A. S. (2026). AarogyaSathi: Design and preliminary evaluation of a multilingual public health information chatbot [Manuscript submitted for publication]. Bhagwan Parshuram Institute of Technology, Delhi.

Laranjo, L., Dunn, A. G., Tong, H. L., Kocaballi, A. B., Chen, J., Bashir, R., Surian, D., Gallego, B., Chughtai, A. A., & Lau, A. Y. S. (2018). Conversational agents in healthcare: A systematic review. Journal of the American Medical Informatics Association, 25(9), 1248–1258. https://doi.org/10.1093/jamia/ocy072

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In Advances in Neural Information Processing Systems (Vol. 33, pp. 9459–9474). Curran Associates, Inc.

Milne-Ives, M., de Cock, C., Lim, E., Shehadeh, M. H., de Pennington, N., Mole, G., Normando, E., & Meinert, E. (2020). The effectiveness of artificial intelligence conversational agents in health care: Systematic review. Journal of Medical Internet Research, 22(10), Article e20346. https://doi.org/10.2196/20346

Ministry of Health and Family Welfare, Government of India. (2024). Universal Immunization Programme: National immunization schedule for infants, children and pregnant women. National Health Mission. https://nhm.gov.in/New_Updates_2018/NHM_Components/Immunization/report/National_Immunization_Schedule.pdf

National Centre for Disease Control. (2024). Integrated Disease Surveillance Programme (IDSP): Strategic framework and weekly outbreak reporting. Directorate General of Health Services, Ministry of Health and Family Welfare. https://ncdc.mohfw.gov.in/

Nilve, H., Parihar, K., & Nichat, A. (2026). AI-driven public health chatbot for disease awareness. International Journal of Engineering Research & Technology, 15(3), Article IJERTV15IS031825. https://www.ijert.org/

Semigran, H. L., Linder, J. A., Gidengil, C., & Mehrotra, A. (2015). Evaluation of symptom checkers for self diagnosis and triage: Audit study. BMJ, 351, Article h3480. https://doi.org/10.1136/bmj.h3480

World Health Organization. (2020). Managing the COVID-19 infodemic: Promoting healthy behaviours and mitigating the harm from misinformation and disinformation. Joint statement by WHO, UN, UNICEF, UNDP, UNESCO, UNAIDS, ITU, UN Global Pulse, and IFRC. https://www.who.int/news/item/23-09-2020-managing-the-covid-19-infodemic

World Health Organization. (2024). Ethics and governance of artificial intelligence for health: Guidance on large multi-modal models (ISBN 978-92-4-008475-9). World Health Organization. https://www.who.int/publications/i/item/9789240084759

APPENDIX A: PRODUCTION HARDWARE & ENVIRONMENT SPECIFICATIONS

This appendix documents the complete hardware topology, operating system parameters, and runtime environment specifications deployed on Oracle Cloud Infrastructure (OCI) hosting AarogyaSathi:

Table A.1: Oracle Cloud VM Production Hardware and Environment Topology

System Attribute	Configured Production Specification	Operational Function
Cloud Hosting Provider	Oracle Cloud Infrastructure (OCI) - Arm virtual machine (VM.Standard.A1.Flex)	Infrastructure compute & storage host
Compute Shape	VM.Standard.A1.Flex (Ampere Altra Arm64 Processor)	2 OCPUs, 12 GiB Unified Physical RAM
Operating System	Ubuntu 24.04 LTS (GNU/Linux 6.8.0-1017-oracle aarch64)	Base operating system environment
Storage Allocation	50 GB Boot Volume (about 32 GiB free)	OS, PyTorch weights, local SQLite DB
Python Runtime	Python 3.12.3 in dedicated virtualenv (/opt/healthbot/venv)	Application execution runtime
Process Supervision	systemd daemons (healthbot.service, aarogya-tunnel.service)	Process auto-restart & logging
Networking & Ingress	Cloudflare quick tunnel (cloudflared 2026.10.3)	Public HTTPS ingress to localhost:8000
LLM Inference Host	Ollama 0.32.15 (C++ llama.cpp backend)	Quantized Qwen3:1.7b local inference
Memory Utilization	System memory in use at inspection: about 3.3 GiB of 11 GiB (includes the Ollama model)	Single observation on 1 November 2026

APPENDIX B: INTENT CLASSIFICATION TAXONOMY & API DIRECTORY

This appendix provides the formal directory of REST API endpoints exposed by AarogyaSathi and their supported HTTP methods:

Table B.1: AarogyaSathi Production REST API Endpoint Directory

Endpoint Route	HTTP Method	Payload Summary	Functional Description & Output
/api/chat	POST	JSON {message, pincode, session_id}	Returns clinical reply text, detected intent, language, latency, and disclaimer
/api/health	GET	None (Health probe)	Returns server status, loaded models, local Ollama backend, supported languages

/api/stats	GET	None (Telemetry query)	Returns cumulative query counts, intent distribution, active session count in RAM
/api/surveillance/alerts	GET	None	Returns active syndromic cluster anomalies and rolling 7-day baselines
/api/surveillance/seed	POST	None	Inserts demonstration symptom records (demonstration use only)
/webhook/whatsapp	GET / POST	Meta Hub Verify / Graph API JSON	Meta webhook challenge verification (GET) and HMAC-SHA256 message handler (POST)
/webhook/whatsapp/simulate	POST	JSON {from, message}	Simulates inbound WhatsApp interaction without requiring active Meta credentials
/	GET	None (Web browser)	Serves index.html lightweight responsive mobile chat interface
/admin	GET	None (Browser / Admin)	Serves admin.html syndromic surveillance cluster monitoring portal